ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KINH TẾ - LUẬT

KHOA HỆ THỐNG THÔNG TIN

HỌC PHẦN
BIG DATA VÀ ỨNG DỤNG

XÂY DỰNG HỆ THỐNG DỰ ĐOÁN THỜI TIẾT VÀ ĐỀ XUẤT LỰA CHỌN ĐIỂM ĐẾN DU LỊCH THEO THỜI GIAN THỰC SỬ DỤNG BIG DATA

GVHD: TS. Nguyễn Thôn Dã

Mã nhóm: 6

K224060772 - Nguyễn Thị Hồng Diễm

K224060785 – Trần Trung Hiếu

K224060779 – Nguyễn Thị Như Hào

K224060822 – Nguyễn Phương Tuyền

K224060825 – Trần Thị Hiếu Xuân

Tháng 2/2026

LỜI CẢM ƠN

Để hoàn thành đồ án "Xây dựng hệ thống dự đoán thời tiết và đề xuất lựa chọn điểm đến du lịch theo thời gian thực sử dụng Big Data", nhóm 6 chúng em đã nhận được rất nhiều sự hỗ trợ, hướng dẫn và tạo điều kiện từ phía nhà trường cũng như thầy cô bộ môn.

Lời đầu tiên, nhóm thực hiện xin bày tỏ lòng biết ơn sâu sắc nhất đến TS. Nguyễn Thôn Dã. Trong suốt quá trình học tập học phần Big Data và ứng dụng , thầy đã tận tình truyền đạt những kiến thức chuyên môn quý báu, đồng thời đưa ra những chỉ dẫn, định hướng kỹ thuật quan trọng giúp nhóm có thể tiếp cận và triển khai thành công các công nghệ hiện đại như Apache Kafka, Apache Spark và hạ tầng Cloud. Những ý kiến đóng góp của thầy không chỉ giúp đồ án hoàn thiện về mặt nội dung mà còn giúp chúng em rèn luyện tư duy giải quyết vấn đề trong lĩnh vực dữ liệu lớn.

Chúng em cũng xin gửi lời cảm ơn đến Ban lãnh đạo Khoa Hệ thống Thông tin đã tạo điều kiện về cơ sở vật chất và môi trường học tập thuận lợi để chúng em có cơ hội thực hành và triển khai các dự án mang tính thực tiễn cao như đề tài này.

Cuối cùng, dù đã dành nhiều tâm huyết và nỗ lực để hoàn thành hệ thống , nhưng do hạn chế về mặt thời gian và kinh nghiệm thực tế, đồ án chắc chắn sẽ không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp, nhận xét từ thầy để hệ thống ngày càng hoàn thiện và có khả năng ứng dụng thực tế cao hơn trong tương lai.

Chúng em kính chúc thầy nhiều sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp giảng dạy và nghiên cứu của mình.

Trân trọng,

Tập thể nhóm 6

MỤC LỤC

PHẦN 1. NỘI DUNG ĐỒ ÁN 1

CHƯƠNG I: TỔNG QUAN VỀ ĐỀ TÀI 1

1.1. Lý do chọn đề tài 1

1.2. Mục tiêu nghiên cứu 2

1.3. Đối tượng và phạm vi nghiên cứu 2

1.4. Ý nghĩa thực tiễn của đề tài 4

1.5. Bố cục đồ án 5

CHƯƠNG II: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ 6

2.1. Tổng quan về Big Data 6

2.1.1. Định nghĩa và đặc trưng 6V của Big Data 6

2.1.2. So sánh Data Lake và Data Warehouse 7

2.2 Kiến trúc xử lý dữ liệu hiện đại trên nền tảng Cloud 8

2.2.1. Điện toán đám mây (Cloud Computing) & Mô hình IaaS 8

2.2.2. Streaming Data 12

2.2.3. Streaming Data 13

2.2.4. Data Lake & Object Storage 14

2.2.5. Trực quan hóa dữ liệu 15

2.3. Các công cụ và công nghệ sử dụng 18

2.3.1. Hạ tầng: Amazon EC2. 18

2.3.2. Lưu trữ: MinIO (Data Lake). 20

2.3.3. Thu thập & Truyền tải: Apache Kafka. 22

2.3.4. Nền tảng xử lý & phân tích: Apache Spark & PySpark. 24

2.3.5. Trực quan hóa: Streamlit. 25

CHƯƠNG III: MÔ TẢ DỮ LIỆU 28

3.1. Nguồn dữ liệu 28

3.2. Cấu trúc dữ liệu 28

3.3. EDA và tiền xử lý 30

3.3.1. Đánh giá và làm sạch chất lượng dữ liệu 30

3.3.2. Thống kê mô tả và phạm vi địa lý 31

3.3.3. Phân tích đơn biến và phân tích theo thời gian 31

3.3.4. Phân tích đa biến thông qua tương quan 31

CHƯƠNG IV: THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG 33

4.1. Kiến trúc hệ thống 33

4.1.1. Lớp thu thập và truyền tải dữ liệu (Ingestion Layer – Apache Kafka) 33

4.1.2. Lớp xử lý và phân tích dữ liệu (Processing Layer – Apache Spark & PySpark) 34

4.1.3. Lớp lưu trữ dữ liệu (Storage Layer – MinIO Data Lake) 34

4.1.4. Lớp trực quan hóa dữ liệu (Visualization Layer – Streamlit). 35

4.2. Thiết lập môi trường triển khai 35

4.3. Quy trình thực hiện chi tiết 36

4.3.2. Giai đoạn 2: Tích hợp và lưu trữ (Storage - Ghi vào MinIO Bronze Layer) 36

4.3.3. Giai đoạn 3: Xử lý và làm sạch dữ liệu (Processing - Spark Transformations) 37

4.3.4. Giai đoạn 4: Xây dựng và huấn luyện mô hình dự đoán (Machine Learning)..... 38

4.3.5. Giai đoạn 5: Trực quan hóa và báo cáo (Visualization - Report)40

CHƯƠNG V: THỰC NGHIỆM VÀ KẾT QUẢ 44

5.1. Kết quả vận hành hệ thống..... 44

5.2. Kết quả phân tích và dự báo 46

5.2.1. Kết quả phân tích dữ liệu thời tiết..... 47

5.2.2. Kết quả dự báo nhiệt độ 47

5.2.3. Kết quả đề xuất thành phố dựa trên dự báo 48

5.3. Kết quả trực quan hóa 48

5.4. Đánh giá hiệu năng 50

CHƯƠNG VI: KẾT LUẬN VÀ KIẾN NGHỊ 51

6.1. Kết quả đạt được 51

6.2. Hạn chế của đề tài 51

6.3. Hướng phát triển trong tương lai 52

TÀI LIỆU THAM KHẢO 53

PHỤ LỤC 54

PHẦN 2. MÔ TẢ PHÂN CÔNG CÔNG VIỆC TRONG NHÓM 55

DANH MỤC BẢNG

Bảng 1 . So sánh giữa Data Lake và Data Warehouse 8

Bảng 2 . Thông số kỹ thuật các thành phần cho kiến trúc đường dẫn dữ liệu..... 19

Bảng 3 . Mô tả biến/cột..... 28

DANH MỤC HÌNH ẢNH

Hình 1 . Sơ đồ kiến trúc điện toán đám mây 10

Hình 2 . Cơ sở hạ tầng như một dịch vụ (IaaS) 11

Hình 3 . So sánh giữa lưu trữ khối và lưu trữ đối tượng..... 14

Hình 4 . Ví dụ về bản đồ trực quan hóa dữ liệu..... 16

Hình 5 . Các loại biểu đồ trực quan hóa dữ liệu phổ biến 17

Hình 6 . Quy trình ETL với Apache Spark và MinIO 21

Hình 7 . Các thành phần kiến trúc của Kafka 23

Hình 8 . Các mô-đun chức năng của Spark 24

Hình 9 . Thống kê mô tả của tập dữ liệu..... 31

Hình 10 . Bảng điều khiển EDA dữ liệu thời tiết 32

Hình 11 . Trực quan hóa dự đoán nhiệt độ theo ngày..... 41

Hình 12 . Trực quan hóa dự đoán nhiệt độ theo giờ 42

Hình 13 . Kết quả vận hành trên EC2 45

Hình 14 . Kết quả chạy thành công trên Kafka..... 46

Hình 15 . Kết quả vận hàng trên MinIo 46

Hình 16 . Kết quả trực quan hóa 50

PHẦN 1. NỘI DUNG ĐỒ ÁN

CHƯƠNG I: TỔNG QUAN VỀ ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của cách mạng công nghiệp 4.0, các công nghệ như Internet of Things (IoT) và Trí tuệ nhân tạo (AI) ngày càng được ứng dụng rộng rãi để giải quyết các vấn đề phức tạp trong thực tế. Lĩnh vực du lịch thông minh đang trở thành xu hướng tất yếu, trong đó việc cá nhân hóa trải nghiệm và cung cấp hỗ trợ quyết định theo thời gian thực cho du khách đóng vai trò then chốt.

Dữ liệu thời tiết là yếu tố ảnh hưởng trực tiếp và sâu sắc đến hành vi, kế hoạch và sự hài lòng của khách du lịch. Tuy nhiên, dữ liệu thời tiết từ các cảm biến IoT và dịch vụ API thường có khối lượng lớn, tốc độ cao và được tạo ra liên tục, đòi hỏi các hệ thống xử lý mạnh mẽ có khả năng phân tích tức thời và trích xuất thông tin chi tiết. Hơn nữa, việc tích hợp dữ liệu thời tiết thời gian thực với thông tin điểm đến để đưa ra các đề xuất phù hợp vẫn là một thách thức về mặt kỹ thuật và thuật toán.

Xuất phát từ thực tế này, nhóm nghiên cứu đã chọn đề tài: "Xây dựng hệ thống dự báo thời tiết và đề xuất điểm đến du lịch theo thời gian thực sử dụng dữ liệu lớn". Dự án này nhằm mục đích nghiên cứu và xây dựng một quy trình xử lý dữ liệu hoàn chỉnh, áp dụng các công nghệ dữ liệu lớn hiện đại như Apache Kafka và Apache Spark Streaming để thu thập, xử lý và phân tích các luồng dữ liệu thời tiết theo thời gian thực. Đồng thời, dự án tích hợp với cơ sở dữ liệu điểm đến du lịch để tự động đề xuất các điểm đến phù hợp dựa trên điều kiện thời tiết. Thông qua dự án này, nhóm nghiên cứu không chỉ mong muốn chứng minh khả năng xử lý dữ liệu lớn theo thời gian thực mà còn muốn thể hiện giá trị ứng dụng thực tiễn của dữ liệu lớn trong việc tạo ra các dịch vụ thông minh nhằm nâng cao trải nghiệm người dùng trong lĩnh vực du lịch.

Ngoài ra, trong bối cảnh ngành du lịch Việt Nam đang phục hồi và phát triển mạnh mẽ, việc nghiên cứu và ứng dụng các hệ thống hỗ trợ ra quyết định dựa trên dữ liệu thời gian thực thông minh là cần thiết. Những hệ thống như vậy có

thể góp phần tạo lợi thế cạnh tranh và thúc đẩy sự phát triển bền vững của ngành.

1.2. Mục tiêu nghiên cứu

Dựa trên lý do lựa chọn đề tài, đồ án hướng tới các mục tiêu nghiên cứu chính sau:

- Xây dựng hệ thống xử lý dữ liệu thời tiết thời gian thực trên nền tảng Cloud, sử dụng Apache Kafka để thu thập dữ liệu streaming và Apache Spark Streaming (PySpark) làm công cụ xử lý và phân tích cốt lõi.
- Mô phỏng và thu thập dữ liệu thời tiết thời gian thực từ các API mã nguồn mở (như Open-Meteo) cho nhiều địa điểm (thành phố, điểm du lịch) trên khắp Việt Nam, tổ chức dữ liệu thành các luồng liên tục.
- Phân tích và xử lý dữ liệu streaming, bao gồm làm sạch dữ liệu, tính toán các chỉ số thời tiết chính như nhiệt độ trung bình, độ ẩm trung bình theo khu vực/thành phố và thực hiện các dự đoán ngắn hạn đơn giản (ví dụ: xu hướng nhiệt độ trong vài giờ tới).
- Phát triển một cơ chế đề xuất điểm đến thông minh dựa trên dữ liệu thời tiết thời gian thực. Hệ thống sẽ so sánh điều kiện thời tiết giữa các địa điểm và đề xuất những điểm đến phù hợp nhất dựa trên các tiêu chí đã định trước (ví dụ: tìm những địa điểm có thời tiết nắng ráo, nhiệt độ dễ chịu hoặc tránh những nơi đang có mưa).
- Đánh giá hiệu năng và tính khả thi của hệ thống bằng cách đo độ trễ xử lý, khả năng mở rộng và độ chính xác của các đề xuất được tạo ra, từ đó tạo nền tảng cho việc phát triển ứng dụng thực tiễn.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu:

- Dữ liệu thời tiết thời gian thực: Dữ liệu được thu thập từ API Open-Meteo (https://open-meteo.com (Phần mềm này mô phỏng luồng dữ liệu từ các điểm quan sát ảo tại các thành phố du lịch trọng điểm trên khắp Việt Nam. Các thông số chính bao gồm: nhiệt độ, độ ẩm, mã thời tiết, lượng mưa và tốc độ gió.)

- Dữ liệu điểm đến du lịch: Một tập dữ liệu tĩnh chứa thông tin về các thành phố du lịch mục tiêu, bao gồm: tên thành phố, tọa độ địa lý (vĩ độ, kinh độ), khu vực và quan trọng nhất, các tiêu chí thời tiết lý tưởng (phạm vi nhiệt độ lý tưởng, điều kiện mưa/nắng ưa thích) để phục vụ thuật toán đề xuất.
- Hệ thống xử lý dữ liệu thời gian thực và đề xuất: Hệ thống sử dụng Apache Kafka làm trung tâm xử lý tin nhắn để thu thập và phân phối luồng dữ liệu thời tiết. Apache Spark Streaming (PySpark) đóng vai trò là công cụ xử lý chính, thực hiện: làm sạch dữ liệu, dự đoán xu hướng thời tiết ngắn hạn (sử dụng các phương pháp dựa trên quy tắc đơn giản hoặc thống kê), và quan trọng nhất là thực thi logic đề xuất điểm đến bằng cách kết hợp dữ liệu thời tiết thời gian thực với các tiêu chí thành phố được xác định trước.

Phạm vi nghiên cứu:

- Phạm vi chức năng: Nghiên cứu tập trung vào việc xây dựng một hệ thống xử lý dữ liệu thời gian thực hoàn chỉnh để:
    - Dự đoán/ước tính xu hướng thời tiết ngắn hạn (ví dụ: nhiệt độ sẽ tăng hay giảm trong vài giờ tới) cho các địa điểm cụ thể.
    - Đề xuất thành phố du lịch phù hợp nhất bằng cách so sánh điều kiện thời tiết thực tế giữa các địa điểm với các tiêu chí lý tưởng đã được xác định trước.
- Phạm vi dữ liệu: Dữ liệu thời tiết được sử dụng là dữ liệu thời gian thực (hoặc được mô phỏng dưới dạng luồng liên tục) từ API Open-Meteo, tập trung vào các thành phố du lịch tiêu biểu trên khắp Việt Nam (ví dụ: Hà Nội, Đà Nẵng, Hải Phòng, Cần Thơ).
- Phạm vi công nghệ: Nghiên cứu này không bao gồm việc triển khai phần cứng IoT thực tế (cảm biến vật lý) hoặc phát triển đầy đủ ứng dụng giao diện người dùng (web/di động). Trọng tâm chính là kiến trúc hệ thống phía máy chủ, bao gồm: bộ tạo dữ liệu, cụm Kafka, ứng dụng

Spark Streaming và cơ sở dữ liệu/API để lưu trữ/truy xuất kết quả đề xuất.

- Phạm vi quy mô và hiệu năng: Khối lượng dữ liệu (số tin nhắn mỗi giây) và cấu hình hệ thống được thiết lập ở mức vừa phải, phù hợp để chạy trên môi trường ảo hóa (Docker, máy ảo đơn) cho mục đích giáo dục, nghiên cứu và chứng minh tính khả thi của kiến trúc được đề xuất.
- Phạm vi thuật toán: Phân tích và dự đoán sẽ sử dụng các phương pháp thống kê cơ bản, logic dựa trên quy tắc, kết hợp với các thuật toán Học máy đơn giản nhưng hiệu quả cho các nhiệm vụ cơ bản. Tuy nhiên, dự án sẽ không đi sâu vào việc xây dựng, huấn luyện và tối ưu hóa các mô hình học máy quá phức tạp, đặc biệt là các mô hình Học sâu yêu cầu lượng dữ liệu khổng lồ và sức mạnh tính toán cao (ví dụ: các mạng LSTM phức tạp, mạng CNN). Dự án cũng không tập trung vào việc phát triển các hệ thống đề xuất cá nhân hóa phức tạp dựa trên lịch sử người dùng. Trọng tâm chính vẫn là chứng minh tính khả thi của kiến trúc xử lý luồng dữ liệu thời gian thực với các kỹ thuật phân tích cơ bản và học máy đơn giản.

1.4. Ý nghĩa thực tiễn của đề tài

Nghiên cứu này mang lại ý nghĩa thực tiễn quan trọng cho các lĩnh vực du lịch thông minh và công nghệ thông tin, như được trình bày chi tiết dưới đây:

- Ứng dụng thực tiễn của dữ liệu lớn trong du lịch: Dự án này chứng minh cách thức tận dụng các công nghệ xử lý dữ liệu trực tuyến để tạo ra các dịch vụ giá trị gia tăng, hỗ trợ trực tiếp cho việc ra quyết định của khách du lịch.
- Nâng cao trải nghiệm du lịch: Ứng dụng cung cấp công cụ đề xuất điểm đến thông minh, linh hoạt dựa trên điều kiện thời tiết thực tế, giúp du khách tối ưu hóa kế hoạch du lịch và tránh những bất tiện do thời tiết gây ra.
- Nền tảng cho dịch vụ du lịch cá nhân hóa: Hệ thống này có thể đóng vai trò là nền tảng để phát triển các ứng dụng tiên tiến hơn, chẳng hạn như

gợi ý lịch trình và đề xuất hoạt động phù hợp với sở thích cá nhân và điều kiện thời tiết hiện hành.

- Tài liệu tham khảo quý giá: Tài liệu này cung cấp mô hình tham khảo cụ thể cho kiến trúc hệ thống dữ liệu lớn tích hợp xử lý thời gian thực với các công cụ đề xuất. Đây là nguồn tài liệu hữu ích cho sinh viên, nhà nghiên cứu và các nhà phát triển công nghệ trong lĩnh vực du lịch.
- Đóng góp vào phát triển du lịch thông minh tại Việt Nam: Dự án khuyến khích việc ứng dụng công nghệ số và phân tích dữ liệu lớn để nâng cao chất lượng dịch vụ du lịch, từ đó tăng cường khả năng cạnh tranh và sức hấp dẫn của các điểm đến du lịch Việt Nam.

1.5. Bố cục đồ án

Cấu trúc của đồ án được chia thành các chương như sau:

- Chương 1: Giới thiệu về đề tài
- Chương 2: Cơ sở lý thuyết và công nghệ
- Chương 3: Mô tả dữ liệu
- Chương 4: Thiết kế và triển khai hệ thống
- Chương 5: Thực nghiệm và kết quả
- Chương 6: Kết luận và kiến nghị

CHƯƠNG II: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. Tổng quan về Big Data

Trong bối cảnh sự phát triển mạnh mẽ của công nghệ số, dữ liệu được tạo ra với tốc độ ngày càng nhanh và quy mô ngày càng lớn. Các nguồn dữ liệu đến từ mạng xã hội, hệ thống IoT, giao dịch trực tuyến, thiết bị di động và nhiều hệ thống thông minh khác đã vượt quá khả năng xử lý của các hệ thống dữ liệu truyền thống. Từ đó, khái niệm Big Data ra đời nhằm mô tả và giải quyết các bài toán liên quan đến dữ liệu có quy mô lớn, đa dạng và được tạo ra liên tục.

Big Data đóng vai trò quan trọng trong nhiều lĩnh vực như kinh doanh, tài chính, y tế, giao thông và môi trường thông minh. Đặc biệt, trong các hệ thống IoT, Big Data cho phép thu thập và phân tích dữ liệu thời gian thực từ cảm biến, hỗ trợ dự báo, ra quyết định và tối ưu hóa hoạt động của hệ thống. Vì vậy, việc hiểu rõ bản chất, đặc trưng và kiến trúc lưu trữ của Big Data là nền tảng quan trọng cho các nghiên cứu và ứng dụng thực tế.

2.1.1. Định nghĩa và đặc trưng 6V của Big Data

2.1.1.1. Định nghĩa Big Data

Big Data là thuật ngữ dùng để chỉ tập hợp dữ liệu có kích thước rất lớn, tốc độ sinh dữ liệu cao và đa dạng về định dạng, vượt quá khả năng lưu trữ và xử lý của các hệ quản trị cơ sở dữ liệu truyền thống. Big Data yêu cầu các công nghệ và kiến trúc đặc biệt để thu thập, lưu trữ, xử lý và phân tích dữ liệu nhằm khai thác giá trị từ dữ liệu.

Trong bối cảnh IoT, Big Data thường xuất hiện dưới dạng dữ liệu streaming, được tạo ra liên tục từ các cảm biến và thiết bị thông minh, đòi hỏi khả năng xử lý theo thời gian thực.

2.1.1.2. Đặc trưng 6V của Big Data

Big Data thường được mô tả thông qua 6 đặc trưng chính, còn gọi là mô hình 6V, bao gồm:

- Volume (Khối lượng): Dữ liệu có kích thước rất lớn, có thể lên đến hàng terabyte hoặc petabyte. Trong hệ thống IoT, số lượng lớn cảm biến hoạt động liên tục tạo ra một khối lượng dữ liệu khổng lồ theo thời gian.

- Velocity (Tốc độ): Dữ liệu được tạo ra và truyền đi với tốc độ rất cao, đặc biệt là dữ liệu thời gian thực (real-time). Ví dụ, dữ liệu thời tiết từ cảm biến IoT được gửi liên tục từng giây, yêu cầu hệ thống phải xử lý nhanh chóng để đảm bảo tính kịp thời.
- Variety (Đa dạng): Dữ liệu Big Data có nhiều định dạng khác nhau như dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc. Trong IoT, dữ liệu có thể bao gồm số liệu đo đạc, log hệ thống, dữ liệu vị trí và metadata.
- Veracity (Độ tin cậy): Dữ liệu có thể chứa nhiễu, sai lệch hoặc thiếu chính xác do lỗi cảm biến, mất kết nối hoặc môi trường đo. Do đó, cần có các bước làm sạch và kiểm tra dữ liệu trước khi phân tích.
- Value (Giá trị): Giá trị của Big Data không nằm ở bản thân dữ liệu, mà ở khả năng khai thác và phân tích dữ liệu để tạo ra thông tin hữu ích, hỗ trợ ra quyết định và dự báo.
- Variability (Biến động): Dữ liệu có thể thay đổi về cấu trúc, tốc độ và ý nghĩa theo thời gian. Trong dữ liệu thời tiết, các giá trị đo có thể biến động mạnh theo mùa, khu vực hoặc điều kiện môi trường.

2.1.2. So sánh Data Lake và Data Warehouse

Trong hệ sinh thái Big Data, Data Lake và Data Warehouse là hai mô hình lưu trữ dữ liệu phổ biến, mỗi mô hình có mục đích và đặc điểm riêng.

2.1.2.1. Data Warehouse

Data Warehouse là hệ thống lưu trữ dữ liệu có cấu trúc, được thiết kế để phục vụ cho các hoạt động phân tích và báo cáo. Dữ liệu trước khi đưa vào Data Warehouse phải được làm sạch, chuẩn hóa và chuyển đổi theo một schema cố định (schema-on-write).

Đặc điểm chính của Data Warehouse:

- Dữ liệu có cấu trúc rõ ràng.
- Schema được xác định trước.
- Phù hợp với báo cáo, phân tích truyền thống (BI).
- Hiệu suất truy vấn cao.
- Khó mở rộng với dữ liệu phi cấu trúc hoặc streaming.

2.1.2.2. Data Lake

Data Lake là mô hình lưu trữ dữ liệu thô, cho phép lưu trữ mọi loại dữ liệu mà không cần định nghĩa schema ngay từ đầu (schema-on-read). Data Lake rất phù hợp với Big Data và dữ liệu IoT, nơi dữ liệu được tạo ra liên tục và đa dạng.

Đặc điểm chính của Data Lake:

- Lưu trữ dữ liệu ở dạng thô.
- Hỗ trợ dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc.
- Linh hoạt trong phân tích và xử lý.
- Dễ mở rộng, chi phí thấp.
- Phù hợp với dữ liệu streaming và machine learning.

2.1.2.3. Bảng so sánh Data Lake và Data Warehouse:

Bảng 1. So sánh giữa Data Lake và Data Warehouse

Tiêu chí	Data Warehouse	Data Lake
Data type	Structured	All types of data
Schema	Schema-on-write	Schema-on-read
Flexibility	Low	High
Scalability	Limited	Very good
Suitable for IoT	Not optimal	Very suitable
Storage cost	High	Low

2.2 Kiến trúc xử lý dữ liệu hiện đại trên nền tảng Cloud

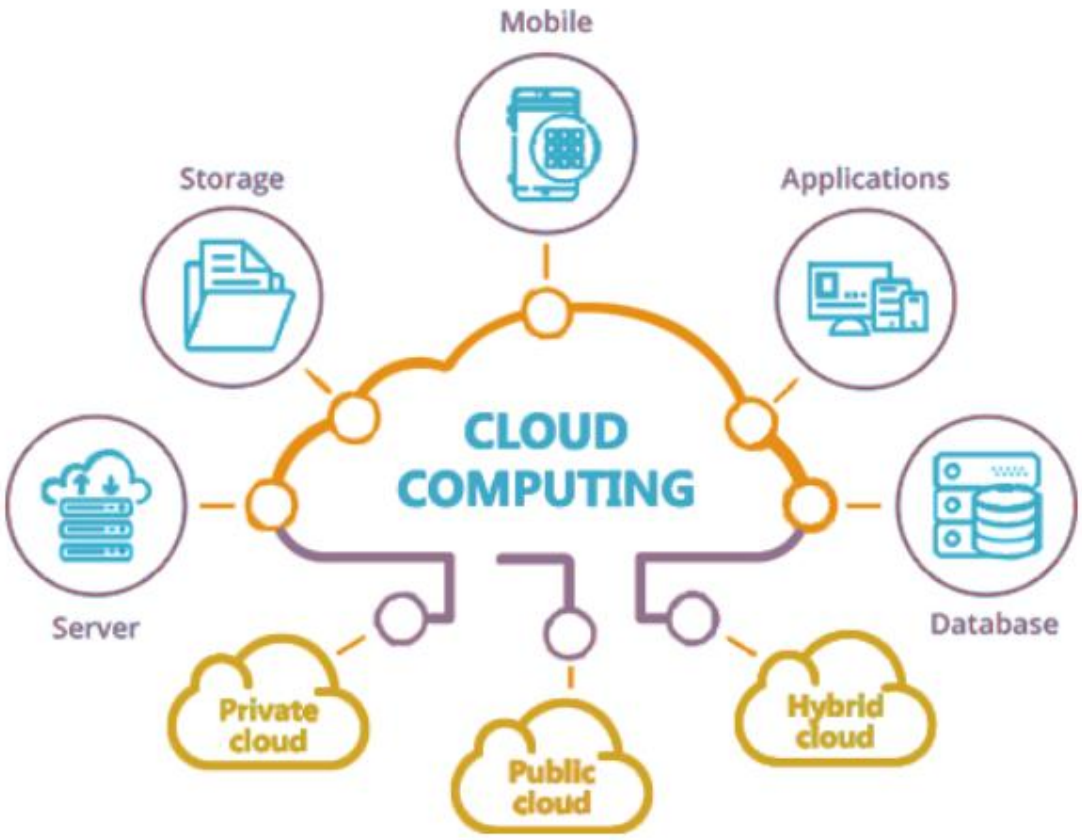
2.2.1. Điện toán đám mây (Cloud Computing) & Mô hình IaaS

2.2.1.1. Điện toán đám mây

Cloud Computing (điện toán đám mây) là thuật ngữ dùng để mô tả mô hình cung cấp và phân phối các tài nguyên công nghệ thông tin theo nhu cầu của người dùng thông qua kết nối Internet. Các tài nguyên này có thể bao gồm máy chủ, hệ thống mạng, năng lực tính toán, lưu trữ dữ liệu và các dịch vụ liên quan

khác. Kiến trúc của điện toán đám mây được xây dựng dựa trên một số thành phần cơ bản như sau:

- Trung tâm dữ liệu: Là nơi tập trung hệ thống máy chủ, thiết bị lưu trữ và các thiết bị mạng, đảm nhiệm chức năng lưu trữ, xử lý và bảo mật dữ liệu cho doanh nghiệp. Trung tâm dữ liệu được vận hành từ xa và thuộc quyền sở hữu, quản lý của các nhà cung cấp dịch vụ điện toán đám mây.
- Kết nối mạng: Bao gồm các mô hình mạng diện rộng (WAN), mạng phân phối nội dung (CDN), mạng định nghĩa bằng phần mềm (SDN) cùng với các bộ cân bằng tải. Các thành phần này đóng vai trò kết nối giao diện người dùng (front-end) với hệ thống tài nguyên phía sau (back-end), đảm bảo việc truyền tải dữ liệu diễn ra ổn định, nhanh chóng và hiệu quả.
- Công nghệ ảo hóa: Là nền tảng cốt lõi của điện toán đám mây, cho phép ảo hóa các thành phần hạ tầng như máy chủ và hệ điều hành thông qua phần mềm chuyên dụng. Việc áp dụng công nghệ ảo hóa giúp hệ thống hoạt động liên tục, hạn chế gián đoạn khi phần cứng vật lý gặp sự cố, đồng thời tối ưu hóa việc khai thác và sử dụng tài nguyên tại các trung tâm dữ liệu.



Hình 1. Sơ đồ kiến trúc điện toán đám mây

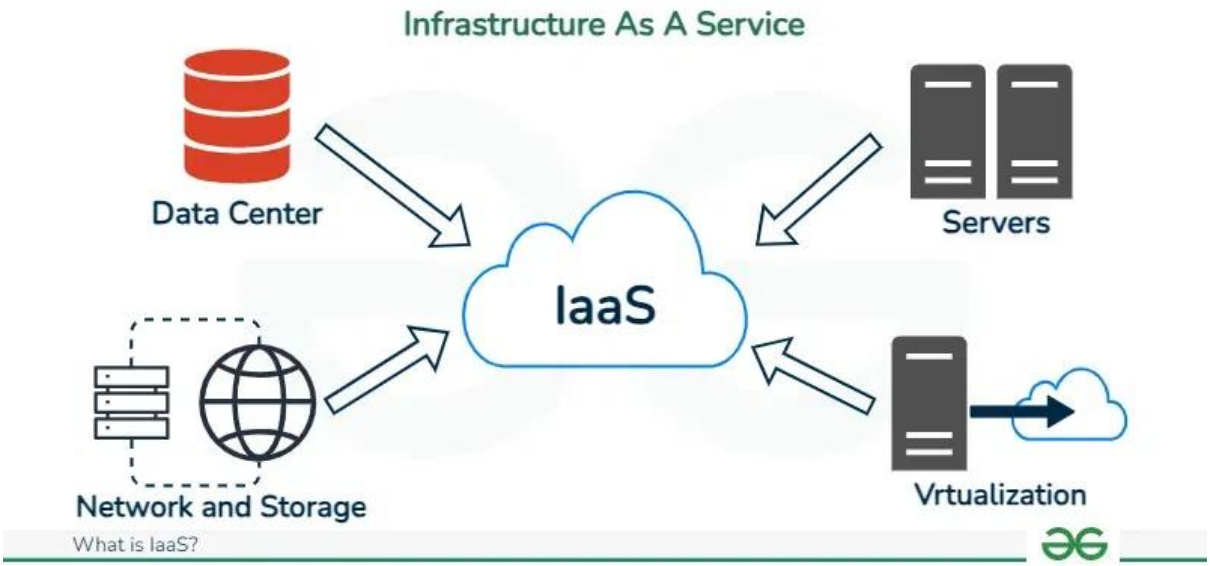
Cloud Computing mang lại nhiều lợi ích quan trọng cho doanh nghiệp, bao gồm:

- Tăng cường khả năng bảo mật: Điện toán đám mây tích hợp các cơ chế bảo mật đa tầng như mã hóa dữ liệu, xác thực hai yếu tố, tường lửa và kiểm soát quyền truy cập, giúp bảo vệ dữ liệu trước các mối đe dọa trên không gian mạng. Nhờ đó, doanh nghiệp có thể giảm thiểu rủi ro tấn công và đánh cắp dữ liệu nội bộ.
- Tiết kiệm chi phí: Thay vì phải đầu tư lớn cho việc mua sắm, nâng cấp và bảo trì phần cứng, phần mềm, doanh nghiệp có thể sử dụng tài nguyên đám mây theo mô hình trả phí dựa trên mức độ sử dụng thực tế, từ đó tối ưu hóa ngân sách đầu tư CNTT.
- Quản lý và vận hành hiệu quả: Cloud Computing cho phép doanh nghiệp truy cập, giám sát và quản lý tài nguyên từ xa thông qua các giao

diện quản trị trực tuyến, giúp giảm bớt gánh nặng trong công tác quản trị và vận hành hệ thống.

- Tính linh hoạt và khả năng mở rộng cao: Doanh nghiệp có thể dễ dàng mở rộng hoặc thu hẹp tài nguyên theo nhu cầu sử dụng trong thời gian ngắn, góp phần rút ngắn thời gian triển khai và nâng cao hiệu quả vận hành hạ tầng.
- Đảm bảo hiệu năng hệ thống: Thông qua việc tích hợp các mô hình kết nối mạng hiện đại cùng bộ cân bằng tải, điện toán đám mây giúp giảm độ trễ mạng, tối ưu tốc độ truyền tải dữ liệu giữa các ứng dụng và thiết bị, từ đó nâng cao hiệu suất và độ ổn định của hệ thống.

2.2.1.2. Cơ sở hạ tầng như một dịch vụ (IaaS)



Hình 2. Cơ sở hạ tầng như một dịch vụ (IaaS)

IaaS cung cấp quyền truy cập theo yêu cầu vào cơ sở hạ tầng trên đám mây, bao gồm các máy chủ, dung lượng lưu trữ và tài nguyên mạng. Người dùng có thể cung cấp, cấu hình và sử dụng các tài nguyên này như phần cứng on-premises. Sự khác biệt nằm ở việc nhà cung cấp dịch vụ đám mây có trách nhiệm lưu trữ, quản lý và bảo trì phần cứng và tài nguyên máy tính trong các trung tâm dữ liệu. Người dùng IaaS truy cập các tài nguyên này thông qua kết nối Internet và thanh toán theo mô hình đăng ký hoặc mô hình thanh toán theo mức sử dụng (pay-as-you-go).

2.2.2. Streaming Data

Streaming data là loại dữ liệu được truyền đi theo khối lượng lớn và liên tục, giúp các tổ chức xử lý nhanh chóng và hiệu quả. Với Streaming data, các doanh nghiệp có thể theo dõi và phân tích các thông tin quan trọng ngay lập tức, giúp họ đưa ra các quyết định đúng đắn và kịp thời.

Đặc điểm của Streaming Data:

- Có tính quan trọng theo thứ tự thời gian: Mỗi thành phần trong Streaming data đều có dấu thời gian và có tầm quan trọng khác nhau theo thời gian. Ví dụ, khi xây dựng một ứng dụng đưa ra đề xuất về nhà hàng dựa trên vị trí hiện tại của người dùng, việc xử lý dữ liệu vị trí địa lý theo thời gian thực là rất quan trọng để đảm bảo tính chính xác và hữu ích của thông tin.
- Truyền liên tục: Streaming data cũng phải được truyền liên tục, tức là không có bắt đầu hay kết thúc rõ ràng. Dữ liệu sẽ được thu thập và truyền đi mà không ngừng, như là nhật ký hoạt động của máy chủ hoặc dữ liệu cảm biến từ thiết bị IoT. Điều này đặt ra thách thức về khả năng xử lý dữ liệu liên tục một cách hiệu quả và đáng tin cậy.
- Tính duy nhất: Việc lặp lại quá trình truyền dữ liệu có thể gặp khó khăn do tính nhạy cảm về thời gian. Để xử lý dữ liệu luồng một cách chính xác, hệ thống phải có khả năng xử lý dữ liệu theo thời gian thực và đảm bảo tính nhất quán của thông tin.
- Không đồng nhất: Dữ liệu có thể được truyền ở nhiều định dạng khác nhau như JSON, Avro, CSV và chứa các loại dữ liệu đa dạng. Hệ thống xử lý luồng phải có khả năng xử lý những định dạng và loại dữ liệu khác nhau một cách linh hoạt và hiệu quả.
- Tính không hoàn hảo: Streaming data không hoàn hảo và có thể gặp phải những lỗi tạm thời từ nguồn dữ liệu. Dẫn đến việc dữ liệu bị hỏng hoặc thiếu, đặt ra thách thức trong việc đảm bảo tính nhất quán và chính xác của thông tin. Việc xây dựng logic xác thực dữ liệu trong hệ thống

phân tích và xử lý luồng là cần thiết để giảm thiểu các lỗi và đảm bảo chất lượng dữ liệu.

2.2.3. Streaming Data

Streaming data là loại dữ liệu được truyền đi theo khối lượng lớn và liên tục, giúp các tổ chức xử lý nhanh chóng và hiệu quả. Với Streaming data, các doanh nghiệp có thể theo dõi và phân tích các thông tin quan trọng ngay lập tức, giúp họ đưa ra các quyết định đúng đắn và kịp thời.

Streaming data có những đặc điểm sau:

- Có tính quan trọng theo thứ tự thời gian: Mỗi thành phần trong Streaming data đều có dấu thời gian và có tầm quan trọng khác nhau theo thời gian. Ví dụ, khi xây dựng một ứng dụng đưa ra đề xuất về nhà hàng dựa trên vị trí hiện tại của người dùng, việc xử lý dữ liệu vị trí địa lý theo thời gian thực là rất quan trọng để đảm bảo tính chính xác và hữu ích của thông tin.
- Truyền liên tục: Streaming data cũng phải được truyền liên tục, tức là không có bắt đầu hay kết thúc rõ ràng. Dữ liệu sẽ được thu thập và truyền đi mà không ngừng, như là nhật ký hoạt động của máy chủ hoặc dữ liệu cảm biến từ thiết bị IoT. Điều này đặt ra thách thức về khả năng xử lý dữ liệu liên tục một cách hiệu quả và đáng tin cậy.
- Tính duy nhất: Việc lặp lại quá trình truyền dữ liệu có thể gặp khó khăn do tính nhạy cảm về thời gian. Để xử lý dữ liệu luồng một cách chính xác, hệ thống phải có khả năng xử lý dữ liệu theo thời gian thực và đảm bảo tính nhất quán của thông tin.
- Không đồng nhất: Dữ liệu có thể được truyền ở nhiều định dạng khác nhau như JSON, Avro, CSV và chứa các loại dữ liệu đa dạng. Hệ thống xử lý luồng phải có khả năng xử lý những định dạng và loại dữ liệu khác nhau một cách linh hoạt và hiệu quả.
- Tính không hoàn hảo: Streaming data không hoàn hảo và có thể gặp phải những lỗi tạm thời từ nguồn dữ liệu. Dẫn đến việc dữ liệu bị hỏng hoặc thiếu, đặt ra thách thức trong việc đảm bảo tính nhất quán và chính

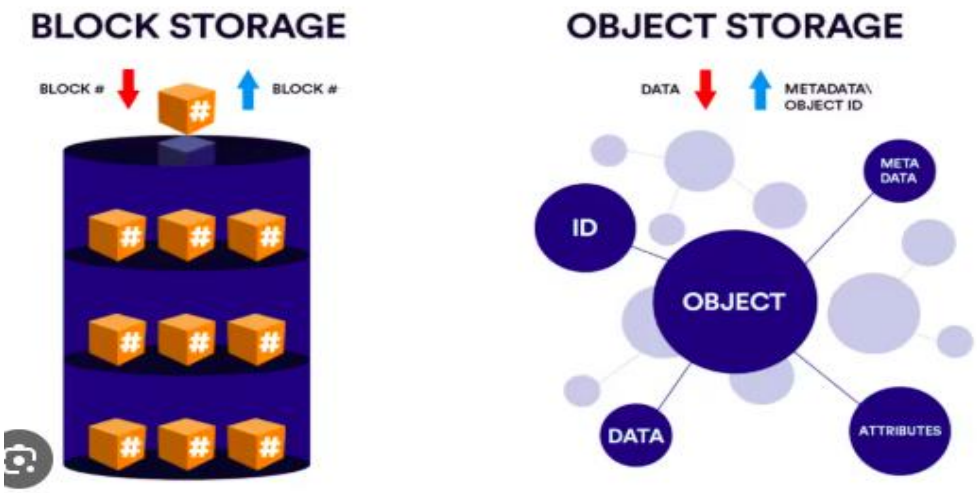
xác của thông tin. Việc xây dựng logic xác thực dữ liệu trong hệ thống phân tích và xử lý luồng là cần thiết để giảm thiểu các lỗi và đảm bảo chất lượng dữ liệu.

2.2.4. Data Lake & Object Storage

Data Lake là một hệ thống lưu trữ dữ liệu phi cấu trúc, cho phép lưu trữ mọi loại dữ liệu từ các nguồn khác nhau. Nó cho phép người dùng truy cập và phân tích các dữ liệu này để tạo ra thông tin hữu ích.

Object Storage (lưu trữ hướng đối tượng), còn được biết đến phổ biến dưới dạng S3 Object Storage, là một kiến trúc lưu trữ dữ liệu hiện đại trong môi trường điện toán đám mây. Khác với mô hình lưu trữ theo cấu trúc thư mục (file storage) hoặc lưu trữ dạng khối (block storage), Object Storage quản lý và lưu trữ dữ liệu dưới dạng các đối tượng (object) độc lập, không phụ thuộc vào cấu trúc phân cấp truyền thống.

Mỗi đối tượng trong Object Storage bao gồm ba thành phần chính: dữ liệu (data), siêu dữ liệu (metadata) và một mã định danh duy nhất, cho phép hệ thống truy xuất, quản lý và mở rộng dữ liệu một cách linh hoạt và hiệu quả. Nhờ khả năng mở rộng cao và cơ chế truy cập thông qua các giao thức chuẩn như S3 (Simple Storage Service), Object Storage đặc biệt phù hợp cho việc lưu trữ khối lượng lớn dữ liệu phi cấu trúc, chẳng hạn như hình ảnh, video, tài liệu và dữ liệu cảm biến trong môi trường đám mây.



Hình 3. So sánh giữa lưu trữ khối và lưu trữ đối tượng

Object Storage sở hữu nhiều tính năng nổi bật bao gồm:

- Phân quyền truy cập: Object Storage cho phép thiết lập các mức độ phân quyền truy cập khác nhau, đảm bảo rằng chỉ những người dùng được ủy quyền mới có thể truy cập vào dữ liệu. Điều này giúp bảo vệ dữ liệu trước các mối đe dọa an ninh và tuân thủ các chính sách bảo mật của doanh nghiệp.
- Lưu trữ và backup tài nguyên: Object Storage có tính năng phục hồi tự động với nhiều bản sao được lưu trữ trên các hệ thống phân tán, giúp đảm bảo an toàn và tính toàn vẹn của dữ liệu, hạn chế rủi ro mất mát dữ liệu ngay cả trong trường hợp xảy ra sự cố.
- Tính năng lưu phiên bản (Versioning): Object Storage cung cấp khả năng lưu phiên bản cho từng đối tượng, cho phép người dùng dễ dàng truy cập các phiên bản cũ của dữ liệu, khôi phục những thay đổi không mong muốn và duy trì lịch sử thay đổi của tài liệu một cách chi tiết.
- Khả năng mở rộng linh hoạt: Khác với các hệ thống lưu trữ truyền thống, Object Storage có khả năng mở rộng gần như vô hạn. Doanh nghiệp có thể dễ dàng bổ sung thêm dữ liệu mà không phải lo lắng về việc điều chỉnh hệ thống hiện tại, giúp tối ưu chi phí và đáp ứng nhu cầu lưu trữ ngày càng tăng.

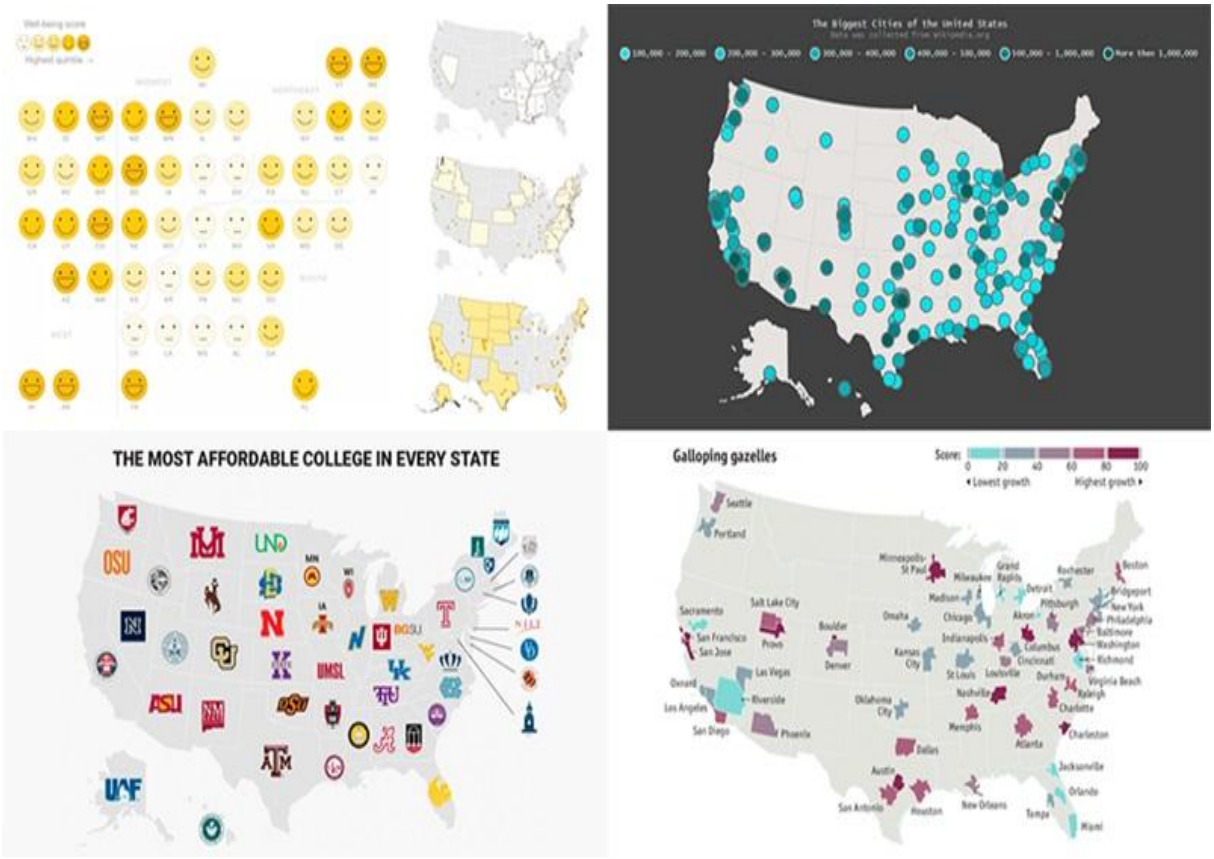
2.2.5. Trực quan hóa dữ liệu

Data Visualization hay còn gọi là trực quan hóa dữ liệu, là quá trình biểu diễn dữ liệu dưới dạng biểu đồ, đồ thị hoặc infographic. Thay vì phải phân tích những bảng số liệu phức tạp, việc sử dụng hình ảnh trực quan cho phép người dùng tiếp cận và nắm bắt thông tin dễ dàng, hiệu quả hơn.

Không chỉ đơn thuần là công cụ hỗ trợ phân tích dữ liệu, Data Visualization còn giúp các doanh nghiệp đưa ra quyết định nhanh chóng và hiệu quả hơn dựa trên những thông tin tổng quan mà họ thấy được từ dữ liệu.

Các loại hình phổ biến của trực quan hóa dữ liệu:

- Map: Map Visualization là phương pháp hiển thị dữ liệu liên quan đến địa lý trên bản đồ. Kỹ thuật này giúp phân tích và phân phối dữ liệu theo khu vực, cho phép doanh nghiệp dễ dàng nhận biết xu hướng theo từng vùng cụ thể.



Hình 4. Ví dụ về bản đồ trực quan hóa dữ liệu

- Chart: Chart Visualization là dạng biểu diễn dữ liệu dưới dạng biểu đồ, đồ thị hoặc bảng biểu. Đây là công cụ trực quan phổ biến nhất, giúp doanh nghiệp dễ dàng so sánh và phân tích thông tin.



Hình 5. Các loại biểu đồ trực quan hóa dữ liệu phổ biến

- Bảng dữ liệu: Tablets (bảng dữ liệu) là một cách thức đơn giản nhưng hiệu quả để hiển thị dữ liệu dưới dạng số liệu thô, không có các yếu tố đồ họa trực quan như dạng Map hay Chart. Thay vì sử dụng hình ảnh sinh động, tablets tập trung vào việc trình bày các con số cụ thể, rõ ràng và chi tiết, giúp người dùng dễ dàng đọc và phân tích.

2.3. Các công cụ và công nghệ sử dụng

2.3.1. Hạ tầng: Amazon EC2.

2.3.1.1. Tổng quan về Amazon EC2

Amazon Elastic Compute Cloud (Amazon EC2) là dịch vụ máy chủ ảo đám mây cốt lõi của Amazon Web Services, cung cấp năng lực điện toán co giãn và linh hoạt trong môi trường đám mây công cộng. Dịch vụ cho phép người dùng khởi tạo, cấu hình và quản lý các máy ảo một cách nhanh chóng mà không cần đầu tư vào cơ sở hạ tầng vật lý truyền thống.

2.3.1.2. Kiến trúc và tính năng nổi bật

EC2 sở hữu một danh mục đa dạng với hơn 750 loại máy ảo (instance) được tối ưu hóa cho các mục đích sử dụng khác nhau:

- Instance Families chuyên biệt:
    - Tối ưu tính toán (Compute Optimized): Dòng C-series với CPU hiệu năng cao, phù hợp cho xử lý batch và ứng dụng khoa học dữ liệu.
    - Tối ưu bộ nhớ (Memory Optimized): Dòng R-series với bộ nhớ RAM lớn, lý tưởng cho cơ sở dữ liệu trong bộ nhớ và phân tích thời gian thực.
    - Tối ưu lưu trữ (Storage Optimized): Dòng I-series tích hợp ổ cứng NVMe, tối ưu cho các tác vụ I/O cao như xử lý luồng dữ liệu và Data Lake.
    - Tính toán tăng tốc (Accelerated Computing): Dòng P và G-series trang bị GPU, phục vụ cho học máy và mô hình hóa đồ họa.
- Đa dạng hệ sinh thái phần cứng: AWS hỗ trợ nhiều kiến trúc vi xử lý, bao gồm Intel, AMD và Arm (Graviton). Điều này cho phép lựa chọn tối ưu về hiệu năng và chi phí. Đặc biệt, EC2 là dịch vụ đám mây duy nhất cung cấp instance Mac và hỗ trợ băng thông mạng lên đến 400 Gbps.
- Linh hoạt trong thanh toán: Người dùng có thể lựa chọn giữa các mô hình On-Demand (trả theo giờ), Reserved Instances (đặt trước để tiết

kiệm chi phí) và Spot Instances (sử dụng tài nguyên dư thừa với giá rất thấp).

2.3.1.3. Lợi ích đối với dự án

Việc lựa chọn Amazon EC2 làm nền tảng hạ tầng cho hệ thống xử lý dữ liệu lớn mang lại những lợi thế sau:

- Tối ưu hiệu năng và chi phí: Khả năng lựa chọn chính xác loại instance phù hợp với từng thành phần giúp tối đa hiệu năng xử lý trong khi kiểm soát chặt chẽ ngân sách. Sự đa dạng của CPU Arm (Graviton) mang lại hiệu suất cao với chi phí thấp hơn cho một số tác vụ.
- Khả năng co giãn tức thời (Elasticity): Hệ thống có thể tự động mở rộng (scale-out) khi khối lượng dữ liệu tăng đột biến và thu hẹp (scale-in) khi không cần thiết. Cơ chế này đảm bảo khả năng đáp ứng và tiết kiệm tài nguyên.
- Triển khai và quản lý tập trung: Toàn bộ cụm máy chủ cho các dịch vụ khác nhau có thể được quản lý thống nhất trong một VPC. Quá trình này có thể tự động hóa triển khai thông qua template CloudFormation hoặc Terraform.
- Độ tin cậy và sẵn sàng cao: Dịch vụ được xây dựng trên các vùng khả dụng (Availability Zones). Kiến trúc này cho phép thiết kế hệ thống chịu lỗi và đảm bảo thời gian hoạt động (uptime) theo SLA lên đến 99.99%.

2.3.1.4. Ứng dụng cụ thể trong kiến trúc hệ thống

Trong kiến trúc của dự án này, các instance EC2 được phân bổ chuyên biệt:

Bảng 2. Thông số kỹ thuật các thành phần cho kiến trúc đường dẫn dữ liệu

Thành phần	Đặc điểm phù hợp	Vai trò trong hệ thống
Apache Kafka	Cân bằng giữa CPU và bộ nhớ, băng thông mạng cao.	Đường ống trung tâm tiếp nhận và phân phối luồng dữ liệu.
Apache Spark	Số lượng vCPU lớn, tối ưu	Xử lý, biến đổi và phân

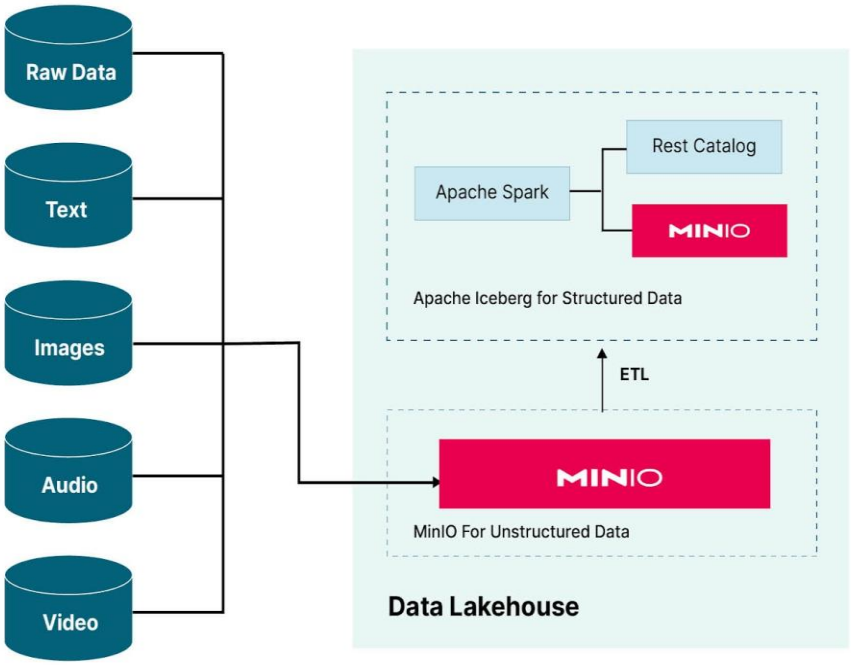
	cho xử lý song song và tính toán phức tạp.	tích dữ liệu theo lô/ luồng.
MinIO (Data Lake)	Lưu trữ cục bộ NVMe dung lượng lớn, I/O cực cao, chi phí thấp trên GB.	Lưu trữ dữ liệu thô và đã xử lý dưới dạng object storage.
Streamlit Dashboard	CPU và bộ nhớ vừa phải, chi phí thấp, phù hợp ứng dụng web nhẹ.	Giao diện trực quan hóa dữ liệu và tương tác người dùng.

Amazon EC2 cung cấp nền tảng hạ tầng lý tưởng cho việc xây dựng một hệ thống xử lý dữ liệu thời gian thực hiện đại. Sự đa dạng của instance types, khả năng co giãn tự động và mô hình giá linh hoạt cho phép thiết kế một kiến trúc tối ưu. Kiến trúc này vừa đảm bảo hiệu năng cao, vừa duy trì hiệu quả về chi phí. Hệ thống cuối cùng có khả năng đáp ứng yêu cầu xử lý khối lượng dữ liệu lớn và phức tạp của dự án.

2.3.2. Lưu trữ: MinIO (Data Lake).

2.3.2.1. Giới thiệu về MinIO

MinIO là giải pháp lưu trữ đối tượng mã nguồn mở với kiến trúc phân tán, được phát triển để đáp ứng nhu cầu lưu trữ dữ liệu phi cấu trúc ở quy mô lớn. Hệ thống này tương thích hoàn toàn với giao thức S3 của Amazon, cho phép tích hợp dễ dàng vào các kiến trúc đám mây hiện có.



Hình 6. Quy trình ETL với Apache Spark và MinIO

2.3.2.2. Đặc điểm kỹ thuật chính

- Chuẩn tương thích S3: Hỗ trợ đầy đủ các API S3, giúp ứng dụng có thể chuyển đổi giữa các hệ thống lưu trữ mà không cần thay đổi mã nguồn.
- Hiệu suất tối ưu: Được thiết kế để đạt hiệu năng cao với khả năng xử lý song song, phù hợp cho các tác vụ đọc/ghi dữ liệu với cường độ lớn.
- Cơ chế bảo mật: Triển khai mã hóa dữ liệu toàn diện, bao gồm mã hóa ở trạng thái lưu trữ và trong quá trình truyền tải, cùng hệ thống xác thực đa lớp.
- Khả năng chịu lỗi: Sử dụng kỹ thuật mã hóa xóa (Erasure Coding) để đảm bảo tính sẵn sàng của dữ liệu ngay cả khi một phần hệ thống gặp sự cố.

2.3.2.3. Vai trò trong hệ thống xử lý dữ liệu

- Tiết kiệm chi phí vận hành: Triển khai trên EC2 giúp giảm đáng kể chi phí so với các dịch vụ lưu trữ được quản lý hoàn toàn, đặc biệt khi xử lý khối lượng dữ liệu ở mức petabyte.

- Kiểm soát toàn diện: Người dùng có quyền kiểm soát hoàn toàn vị trí lưu trữ, chính sách bảo mật và các yêu cầu tuân thủ liên quan đến dữ liệu.
- Tích hợp đa nền tảng: Khả năng tương thích với S3 giúp MinIO làm việc hiệu quả với Apache Spark (qua giao thức s3a://), Apache Kafka Connect, cùng nhiều công cụ phân tích khác.
- Nền tảng Data Lake lý tưởng: Hỗ trợ lưu trữ đa dạng định dạng dữ liệu từ thô đến đã xử lý trong cùng một hệ thống thống nhất.

2.3.2.4. Triển khai thực tế trong dự án

Trong kiến trúc này, MinIO đảm nhận vai trò kho dữ liệu trung tâm:

- Khu vực dữ liệu thô: Lưu trữ dữ liệu nguyên bản thu thập từ các nguồn khác nhau.
- Khu vực dữ liệu đã xử lý: Chứa dữ liệu đã được làm sạch và chuẩn hóa sau khi xử lý.
- Khu vực kết quả phân tích: Lưu trữ các báo cáo, mô hình và dataset cuối cùng phục vụ cho mục đích trực quan hóa.

2.3.2.5. Chiến lược quản lý dữ liệu

- Phân tầng lưu trữ: Dữ liệu thường xuyên truy cập được giữ trên instance i3en với ổ NVMe hiệu suất cao. Dữ liệu ít dùng được chuyển sang các dịch vụ lưu trữ chi phí thấp hơn.
- Tự động hóa vòng đời: Thiết lập chính sách tự động chuyển đổi hoặc xóa dữ liệu dựa trên thời gian và tần suất sử dụng.

2.3.3. Thu thập & Truyền tải: Apache Kafka.

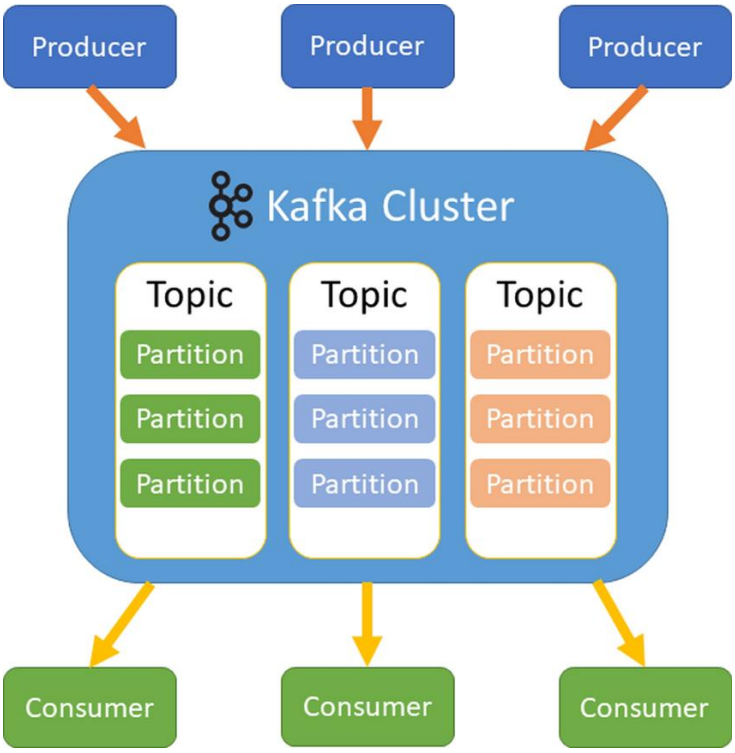
2.3.3.1. Tổng quan về Kafka

Apache Kafka là nền tảng xử lý luồng dữ liệu phân tán, hoạt động theo mô hình nhà xuất bản - người đăng ký (publish-subscribe). Hệ thống này được thiết kế để xử lý khối lượng lớn sự kiện theo thời gian thực với độ trễ thấp.

2.3.3.2. Thành phần cấu trúc

- Nhà sản xuất (Producer): Các ứng dụng hoặc dịch vụ gửi luồng dữ liệu đến Kafka.

- Người tiêu dùng (Consumer): Hệ thống đọc và xử lý dữ liệu từ Kafka.
- Chủ đề (Topic): Phân loại hoặc kênh dữ liệu mà các thông điệp được gửi đến.
- Phân vùng (Partition): Đơn vị chia nhỏ của topic để cho phép xử lý song song.
- Môi giới (Broker): Máy chủ Kafka đơn lẻ trong cụm, chịu trách nhiệm lưu trữ và phân phối dữ liệu.



Hình 7. Các thành phần kiến trúc của Kafka

2.3.3.3. Giá trị mang lại cho dự án

- Đơn giản hóa kiến trúc kết nối: Đóng vai trò làm trung tâm phân phối dữ liệu duy nhất, thay thế mạng lưới kết nối trực tiếp phức tạp giữa các thành phần hệ thống.
- Xử lý luồng dữ liệu tốc độ cao: Có khả năng xử lý hàng triệu sự kiện mỗi giây với thời gian phản hồi ở mức mili giây.
- Đảm bảo độ tin cậy: Dữ liệu được sao lưu trên nhiều broker và có thể lưu giữ trong thời gian dài, hỗ trợ khả năng xử lý lại khi cần thiết.

- Mở rộng linh hoạt: Dễ dàng mở rộng quy mô hệ thống bằng cách bổ sung thêm các broker và phân vùng.

2.3.3.4. Triển khai cụ thể

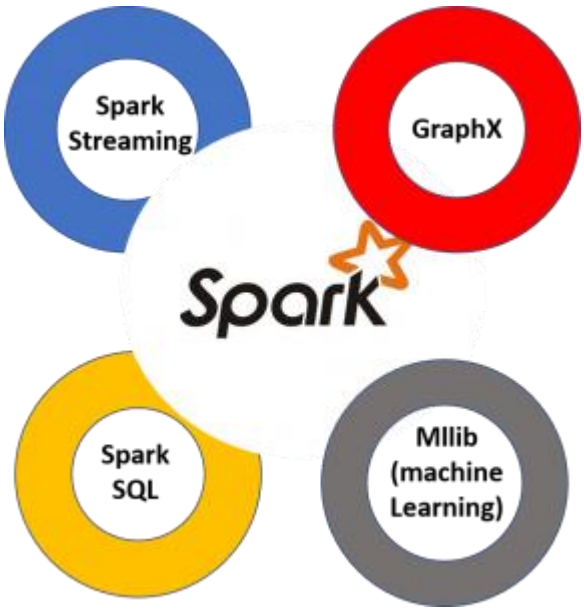
- Luồng tương tác người dùng: Thu thập dữ liệu hành vi từ các trang web và ứng dụng.
- Luồng nhật ký hệ thống: Tập hợp log từ các dịch vụ và ứng dụng phân tán.
- Luồng thiết bị IoT: Tiếp nhận dữ liệu từ các cảm biến và thiết bị thông minh.
- Các job Spark Streaming sẽ đăng ký làm Consumer để xử lý dữ liệu từ các luồng này.

2.3.4. Nền tảng xử lý & phân tích: Apache Spark & PySpark.

2.3.4.1. Giới thiệu về Spark

Apache Spark là framework xử lý dữ liệu phân tán với khả năng xử lý trong bộ nhớ, mang lại hiệu suất vượt trội so với các phương pháp xử lý truyền thống. PySpark là giao diện Python cho Spark, cho phép lập trình viên Python tận dụng sức mạnh của Spark.

2.3.4.2. Các module chức năng



Hình 8. Các mô-đun chức năng của Spark

- Spark SQL: Công cụ xử lý dữ liệu có cấu trúc với hỗ trợ DataFrame và ngôn ngữ truy vấn SQL.
- Structured Streaming: API xử lý luồng dữ liệu được xây dựng trên Spark SQL, cho phép xử lý thời gian thực theo micro-batch.
- MLlib: Thư viện học máy phân tán với các thuật toán được tối ưu cho xử lý quy mô lớn.
- GraphX: Công cụ xử lý đồ thị và tính toán mạng.

2.3.4.3. Lợi thế trong xử lý dữ liệu

- Tăng tốc xử lý: Sử dụng bộ nhớ làm trung gian xử lý chính giúp giảm đáng kể thời gian thực thi so với các phương pháp dựa trên đĩa.
- Thống nhất batch và streaming: Structured Streaming cho phép sử dụng chung code base cho cả xử lý theo lô và xử lý luồng, giảm chi phí phát triển và bảo trì.
- Giao diện lập trình thân thiện: DataFrame API cung cấp cách tiếp cận trực quan để thao tác dữ liệu mà không cần hiểu sâu về lập trình phân tán.
- Hệ sinh thái mở rộng: Kết nối dễ dàng với nhiều nguồn dữ liệu và thư viện Python phổ biến.

2.3.4.4. Ứng dụng thực tiễn

- Quy trình ETL: Đọc dữ liệu từ MinIO, thực hiện các phép biến đổi phức tạp, và ghi kết quả trở lại hệ thống lưu trữ.
- Xử lý thời gian thực: Kết nối trực tiếp với Kafka để xử lý và tổng hợp dữ liệu streaming ngay khi nhận được.
- Phân tích khám phá: Cung cấp môi trường tương tác cho nhà khoa học dữ liệu thông qua PySpark shell và Jupyter notebook.

2.3.5. Trực quan hóa: Streamlit.

2.3.5.1. Khái niệm cơ bản

Streamlit là framework Python chuyên biệt cho việc xây dựng ứng dụng web tương tác trong lĩnh vực khoa học dữ liệu. Công cụ này cho phép chuyển đổi

script Python thành ứng dụng web mà không yêu cầu kiến thức về phát triển front-end.

2.3.5.2. Đặc điểm nổi bật

- Mô hình thực thi tuần tự: Ứng dụng được chạy lại toàn bộ khi có tương tác người dùng, với cơ chế cache thông minh để tối ưu hiệu suất.
- Thành phần tương tác đa dạng: Cung cấp nhiều widget như thanh trượt, nút bấm, hộp chọn để người dùng điều khiển dữ liệu.
- Hiển thị trực quan: Hỗ trợ nhiều định dạng hiển thị từ văn bản, bảng dữ liệu đến các loại biểu đồ khác nhau.
- Tổ chức giao diện linh hoạt: Cho phép sắp xếp thành phần theo cột, tab hoặc thanh bên.

2.3.5.3. Giá trị ứng dụng

- Phát triển nhanh chóng: Biến các phân tích trong notebook thành ứng dụng tương tác hoàn chỉnh trong thời gian ngắn.
- Tập trung vào logic nghiệp vụ: Lập trình viên có thể tập trung vào xử lý dữ liệu mà không cần quan tâm đến các vấn đề giao diện phức tạp.
- Tích hợp sâu với hệ sinh thái Python: Làm việc trực tiếp với Pandas, PySpark và các thư viện trực quan hóa Python phổ biến.
- Triển khai đơn giản: Ứng dụng có thể được đóng gói và triển khai trên nhiều nền tảng khác nhau từ máy chủ vật lý đến đám mây.

2.3.5.4. Triển khai trong hệ thống

- Bảng điều khiển thời gian thực: Hiển thị các chỉ số quan trọng được cập nhật liên tục từ luồng xử lý.
- Báo cáo tương tác: Cho phép người dùng lọc, nhóm và phân tích dữ liệu theo nhiều chiều khác nhau.Giao diện giám sát: Cung cấp cái nhìn tổng quan về trạng thái hoạt động của toàn hệ thống.
- Triển khai mô hình: Tạo giao diện đơn giản để người dùng đầu cuối tương tác với các mô học máy.

2.3.5.5. Tổng kết về kiến trúc tích hợp

- Sự kết hợp của MinIO, Apache Kafka, Apache Spark, và Streamlit trên nền tảng Amazon EC2 tạo thành một kiến trúc xử lý dữ liệu toàn diện. Kiến trúc này hỗ trợ toàn bộ vòng đời dữ liệu từ thu thập, lưu trữ, xử lý đến hiển thị, với khả năng mở rộng linh hoạt và chi phí tối ưu. Mỗi thành phần được lựa chọn dựa trên khả năng đáp ứng yêu cầu cụ thể của hệ thống xử lý dữ liệu lớn thời gian thực.

CHƯƠNG III: MÔ TẢ DỮ LIỆU

3.1. Nguồn dữ liệu

Dữ liệu được thu thập từ API của Open-Meteo (https://open-meteo.com/), một dịch vụ cung cấp dữ liệu thời tiết toàn cầu miễn phí. Đây là dữ liệu mô phỏng luồng dữ liệu thời gian thực, được thiết kế để giả lập luồng dữ liệu liên tục từ một mạng lưới các cảm biến IoT ảo được triển khai tại nhiều thành phố khác nhau trên khắp Việt Nam. Việc sử dụng dữ liệu mô phỏng này cho phép nghiên cứu có được một bộ dữ liệu lớn, đồng nhất, và có cấu trúc rõ ràng để phân tích các hiện tượng khí tượng và xây dựng các mô hình dự báo.

3.2. Cấu trúc dữ liệu

Bộ dữ liệu bao gồm tổng cộng 1,143,360 dòng quan sát và 17 cột thuộc tính, phản ánh các thông số khí tượng cơ bản cùng với thông tin định danh vị trí như: timestamp, temperature, humidity, apparent_temp, weather_code, pressure, rain, wind_speed, wind_direction, wind_gusts, solar_radiation, visibility, cloud_cover, is_day, cùng với metadata về city, region và sensor_id.

Khoảng thời gian dữ liệu: 31/12/1999 17:00 đến 27/01/2026 16:00 (khoảng 26 năm dữ liệu liên tục).

Bảng 3. Mô tả biến/cột

STT	Variable/ Parameter	Unit	Description
1	timestamp	ISO8601	Thời điểm ghi nhận dữ liệu, thường ở định dạng ISO 8601 (YYYY-MM-DD HH:MM:SS), đại diện cho luồng dữ liệu thời gian thực.
2	temperature	°C (°F)	Nhiệt độ không khí, đơn vị đo lường thường là độ Celsius (°C).
3	humidity	%	Độ ẩm tương đối của không khí, được biểu thị bằng phần trăm (%).

4	apparent_temp	°C	Nhiệt độ cảm nhận (Heat Index), kết hợp nhiệt độ và độ ẩm để phản ánh cảm giác thực tế của con người, đơn vị °C.
5	weather_code	WMO Code	Mã số phân loại điều kiện thời tiết tổng quát (ví dụ: 0=Trời quang, 1=Chủ yếu quang, 3=U ám, 45,48=Sương mù, 51,53,55=Mưa phùn, 61,63,65=Mưa rào, 95,96,99=Dông...).
6	pressure	hPa	Áp suất khí quyển tại mực nước biển, đơn vị thường là hectopascal (hPa).
7	rain	mm	Lượng mưa tích lũy trong khoảng thời gian trước đó (ví dụ: giờ trước), đơn vị milimét (mm).
8	wind_speed	km/h	Tốc độ gió trung bình, đơn vị thường là km/h hoặc m/s.
9	wind_direction	°	Hướng gió, biểu thị bằng góc từ 0 đến 360 độ, trong đó 0° là gió Bắc, 90° là gió Đông, 180° là gió Nam, 270° là gió Tây.
10	wind_gusts	km/h	Tốc độ gió giật mạnh nhất trong một khoảng thời gian ngắn, đơn vị tương tự tốc độ gió (km/h hoặc m/s).
11	solar_radiation	km/h	Cường độ bức xạ mặt trời, đơn vị thường là watt trên mét vuông

			(W/m²).
12	visibility	W/m²	Tầm nhìn xa, khoảng cách lớn nhất có thể nhìn thấy các đối tượng, đơn vị thường là mét (m) hoặc kilômét (km).
13	cloud_cover	%	Tỷ lệ phần trăm bầu trời bị che phủ bởi mây (%).
14	is_day	Binary	Cờ chỉ định ban ngày hay ban đêm. Thường là giá trị nhị phân (1 = Ban ngày, 0 = Ban đêm).
15	city	_	Tên thành phố nơi đặt cảm biến IoT ảo (ví dụ: Hà Nội, Đà Nẵng, TP. Hồ Chí Minh).
16	region	_	Tên vùng/ khu vực địa lý của thành phố đó (ví dụ: Đồng bằng sông Hồng, Duyên hải miền Trung, Đông Nam Bộ).
17	sensor_id	_	Định danh duy nhất (ID) của cảm biến IoT ảo. Mỗi thành phố có thể có một hoặc nhiều sensor. Giúp theo dõi và liên kết dữ liệu với một nguồn cụ thể trong mạng lưới.

3.3. EDA và tiền xử lý

3.3.1. Đánh giá và làm sạch chất lượng dữ liệu

- Đã xác định cột "khả năng hiển thị" có 100% giá trị bị thiếu.
- Biện pháp đã thực hiện: Cột này đã bị loại bỏ hoàn toàn khỏi tập dữ liệu để duy trì tính toàn vẹn của dữ liệu và hiệu quả của mô hình.

- Không phát hiện thêm giá trị thiếu đáng kể hoặc giá trị ngoại lệ rõ ràng nào khác trong các trường số còn lại.

3.3.2. Thống kê mô tả và phạm vi địa lý

- Số lượng ghi nhận: 1,143,360 lượt quan sát mỗi giờ.
- Phạm vi thời gian: 26 năm (từ 31/12/1999 đến 27/01/2026).

Biến	Trung bình	Độ lệch chuẩn	Giá trị nhỏ nhất	Giá trị lớn nhất	Số quan sát
temperature	25.05	4.67	4.37	40.15	914304
humidity	81.34	12.84	20.51	100.00	914304
apparent_temp	28.78	6.74	-0.39	48.03	914304
pressure	1009.42	5.90	963.32	1038.88	914304
rain	0.23	0.88	0.00	56.90	914304
wind_speed	8.95	4.75	0.00	78.93	914304
wind_direction	153.67	96.32	0.43	360.00	914304
wind_gusts	19.54	9.26	0.36	150.48	914304
solar_radiation	181.40	260.08	0.00	1029.00	914304
cloud_cover	73.21	33.73	0.00	100.00	914304

Hình 9. Thống kê mô tả của tập dữ liệu

- Phạm vi địa lý: Năm thành phố thuộc ba khu vực:
    - Red River Delta: Ha noi, Hai Phong
    - TP. Hồ Chí Minh (Trung tâm kinh tế lớn nhất)
    - Vùng Nam Trung Bộ: Đà Nẵng
    - Đồng bằng sông Mekong: Cần Thơ

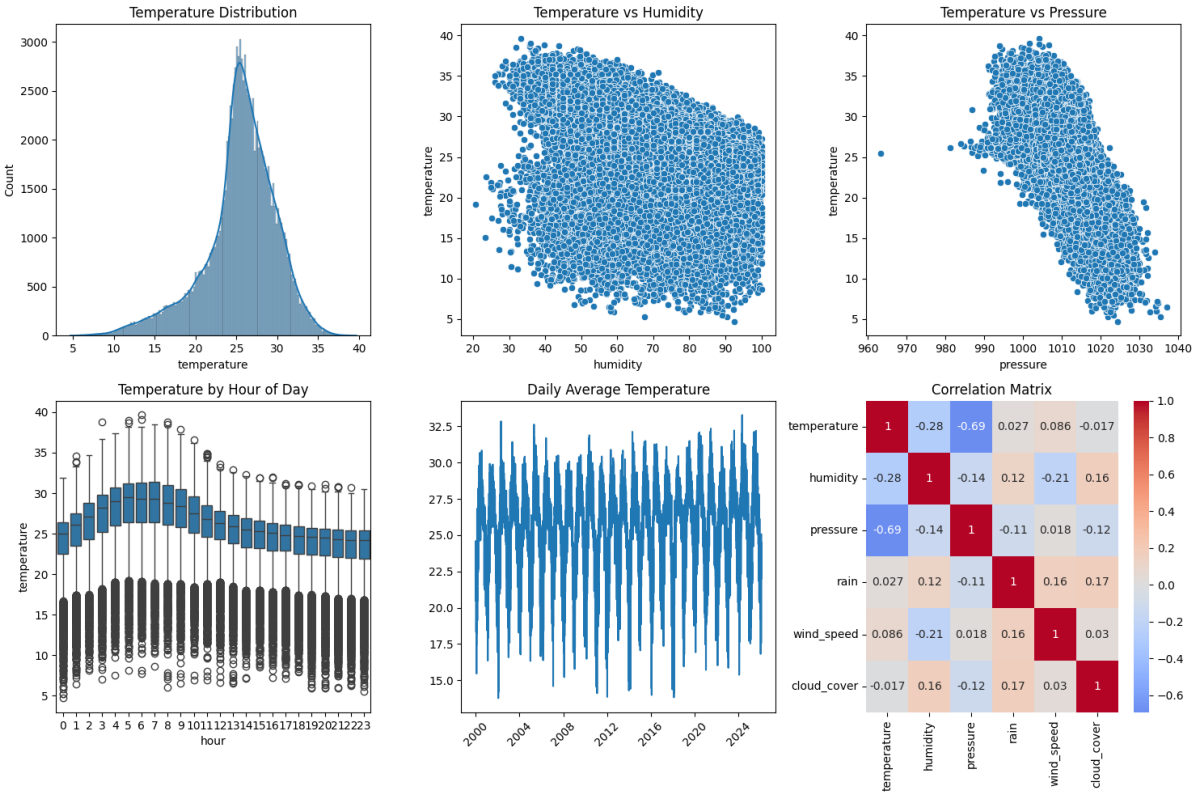
3.3.3. Phân tích đơn biến và phân tích theo thời gian

- Biến số trung tâm - Nhiệt độ: Là mục tiêu chính để phân tích và dự báo. Chu kỳ ngày đêm của nhiệt độ rất rõ rệt, đạt đỉnh điểm vào lúc 14:00-16:00 và đáy vào lúc 05:00-06:00 giờ địa phương.
- Các mô hình chu kỳ: Dữ liệu xác nhận các chu kỳ ngày đêm mạnh mẽ và các chu kỳ theo mùa được nghi ngờ, điều này rất quan trọng đối với các mô hình dự báo chuỗi thời gian.

3.3.4. Phân tích đa biến thông qua tương quan

- Ma trận tương quan đã được tính toán cho tất cả các đặc trưng số.
- Những phát hiện chính:
    - Có mối tương quan nghịch mạnh giữa nhiệt độ và độ ẩm (khoảng -0,7).

- Áp suất có mối tương quan yếu/trung bình với nhiệt độ, thay đổi tùy thuộc vào hệ thống thời tiết.
- Tốc độ gió và lượng mưa cho thấy mối tương quan thuận chiều như dự kiến.
- Ý nghĩa: Mối tương quan mạnh giữa một số đặc điểm (ví dụ: Nhiệt độ/Độ ẩm) sẽ được xem xét để tránh hiện tượng đa cộng tuyến trong mô hình dự đoán (ví dụ: sử dụng phương pháp điều chỉnh hoặc chọn một đặc điểm đại diện).



Hình 10. Bảng điều khiển EDA dữ liệu thời tiết

CHƯƠNG IV: THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

4.1. Kiến trúc hệ thống

Hệ thống được xây dựng dựa trên mô hình đường ống xử lý dữ liệu luồng (Streaming Data Pipeline), một kiến trúc phù hợp với đặc điểm của dữ liệu thời tiết trong các hệ thống Internet of Things (IoT).

Để đáp ứng các yêu cầu trên, kiến trúc hệ thống được thiết kế theo hướng phân lớp chức năng, trong đó mỗi lớp đảm nhận một vai trò riêng biệt nhưng có mối liên kết chặt chẽ với nhau. Cách tiếp cận này cho phép tách biệt rõ ràng các giai đoạn trong vòng đời dữ liệu, từ thu thập, xử lý, lưu trữ đến trực quan hóa và phân tích. Nhờ đó, hệ thống không chỉ dễ dàng trong việc quản lý và bảo trì mà còn thuận lợi cho việc mở rộng hoặc nâng cấp khi yêu cầu nghiệp vụ thay đổi trong tương lai.

4.1.1. Lớp thu thập và truyền tải dữ liệu (Ingestion Layer – Apache Kafka)

Lớp thu thập và truyền tải dữ liệu (Ingestion Layer) được thiết kế nhằm tiếp nhận dữ liệu khí tượng từ nguồn bên ngoài và phân phối dữ liệu đến các thành phần xử lý và lưu trữ trong hệ thống. Trong nghiên cứu này, Apache Kafka được sử dụng làm nền tảng trung gian để điều phối luồng dữ liệu.

Dữ liệu khí tượng được truy vấn từ API theo chu kỳ và được gửi vào hệ thống thông qua các Kafka Producer. Các thông điệp dữ liệu được tổ chức theo các Kafka Topic riêng biệt, giúp phân tách luồng dữ liệu theo từng khu vực và thuận lợi cho việc mở rộng hệ thống. Kafka Consumer đóng vai trò tiếp nhận dữ liệu từ các Topic, thực hiện chuẩn hóa định dạng và chuyển tiếp dữ liệu đến tầng lưu trữ.

Việc sử dụng Apache Kafka trong lớp Ingestion giúp hệ thống đạt được các đặc tính quan trọng như khả năng xử lý dữ liệu theo luồng, độ tin cậy trong truyền tải và khả năng mở rộng khi khối lượng dữ liệu tăng lên. Lớp này đóng vai trò là cầu nối giữa nguồn dữ liệu và các tầng xử lý phía sau trong kiến trúc hệ thống.

4.1.2. Lớp xử lý và phân tích dữ liệu (Processing Layer – Apache Spark & PySpark)

Hệ thống được triển khai trên nền tảng Amazon Web Services (AWS) theo mô hình IaaS, trong đó Apache Kafka và PySpark đóng vai trò trung tâm trong kiến trúc xử lý dữ liệu thời gian thực. Việc sử dụng EC2 giúp hệ thống linh hoạt trong việc cấp phát tài nguyên và dễ dàng mở rộng khi khối lượng dữ liệu tăng lên.

- Kafka Cluster được triển khai trên EC2 instances, mỗi instance đảm nhiệm vai trò Kafka Broker. Cụm Kafka chịu trách nhiệm tiếp nhận dữ liệu thời tiết từ nguồn thu thập và phân phối dữ liệu theo mô hình publish–subscribe cho các tiến trình xử lý phía sau.
- PySpark được sử dụng để triển khai các job xử lý luồng với Spark Structured Streaming, đọc dữ liệu trực tiếp từ Kafka, thực hiện các bước làm sạch, biến đổi, tổng hợp và sinh ra kết quả phân tích. Cụm Spark có khả năng mở rộng số lượng worker để đáp ứng yêu cầu xử lý khi tốc độ dữ liệu đầu vào tăng cao.
- Các thành phần Kafka và PySpark phối hợp chặt chẽ với nhau, trong đó Kafka đảm nhiệm vai trò hàng đợi dữ liệu và truyền dữ liệu theo luồng, còn PySpark đóng vai trò xử lý và phân tích dữ liệu lớn theo thời gian thực. Cách tiếp cận này giúp hệ thống đảm bảo tính mở rộng, khả năng xử lý song song và hiệu năng cao, phù hợp với bài toán phân tích dữ liệu thời tiết liên tục.

Việc triển khai và cấu hình các dịch vụ được thực hiện thủ công hoặc bán tự động trên EC2, đảm bảo hệ thống vận hành ổn định trong môi trường thử nghiệm của đồ án.

4.1.3. Lớp lưu trữ dữ liệu (Storage Layer – MinIO Data Lake)

MinIO được sử dụng làm hệ thống lưu trữ đối tượng và đóng vai trò như một Data Lake cho toàn bộ hệ thống. Dữ liệu được tổ chức theo mô hình phân vùng nhiều lớp (zones) nhằm thuận tiện cho quản lý và phân tích:

- raw-zone/: lưu trữ dữ liệu thô từ Kafka, phục vụ mục đích backup hoặc xử lý lại khi cần.
- processed-zone/: lưu trữ dữ liệu đã qua xử lý từ Spark Streaming dưới định dạng Parquet, được phân vùng theo thời gian (năm/tháng/ngày).
- prediction/: chứa dữ liệu dự đoán từ mô hình dự đoán.

Spark được cấu hình sử dụng Hadoop S3A Connector để kết nối trực tiếp với MinIO, cho phép đọc và ghi dữ liệu một cách hiệu quả.

4.1.4. Lớp trực quan hóa dữ liệu (Visualization Layer – Streamlit)

Lớp hiển thị được xây dựng bằng Streamlit, cung cấp giao diện trực quan cho người dùng theo dõi thông tin thời tiết và kết quả phân tích. Dashboard bao gồm:

- Bảng dữ liệu hiển thị nhiệt độ dự đoán theo từng vị trí.
- Biểu đồ xu hướng nhiệt độ dự đoán theo thời gian.
- Bảng dữ liệu dự đoán cập nhật liên tục từ hệ thống lưu trữ
- Bảng dữ liệu đề xuất thành phố
- Biểu đồ so sánh nhiệt độ dự đoán giữa các vị trí

Ứng dụng Streamlit truy xuất dữ liệu trực tiếp từ MinIO (processed-zone) hoặc từ Kafka để đảm bảo khả năng cập nhật gần thời gian thực.

4.2. Thiết lập môi trường triển khai

Hệ thống được triển khai trên nền tảng Amazon Web Services (AWS) theo mô hình IaaS, tận dụng khả năng mở rộng và linh hoạt của hạ tầng đám mây.

- Kafka: triển khai trên 3 EC2 instances loại t3.medium, sử dụng EBS gp3 để lưu trữ.
- PySpark: bao gồm 1 node Master (m5.large) và 2–4 node Worker (m5.xlarge), có khả năng mở rộng theo tải.
- MinIO: triển khai theo mô hình phân tán trên 4 EC2 instances i3.large, sử dụng ổ NVMe để tối ưu hiệu năng I/O.
- Streamlit: được đóng gói bằng Docker và triển khai trên một EC2 instance t3.small, sử dụng Nginx làm reverse proxy.

4.3. Quy trình thực hiện chi tiết

4.3.1. Giai đoạn 1: Thu thập dữ liệu (Ingestion - Kafka Producer/Consumer)

Giai đoạn thu thập dữ liệu được triển khai nhằm thu thập dữ liệu khí tượng từ nguồn API và truyền tải dữ liệu thông qua hệ thống Apache Kafka. Mục tiêu của giai đoạn này là đảm bảo dữ liệu được thu thập liên tục, có cấu trúc thống nhất và sẵn sàng cho quá trình lưu trữ.

Kafka Producer thực hiện truy vấn dữ liệu khí tượng theo chu kỳ giờ cho năm thành phố gồm Hà Nội, Thành phố Hồ Chí Minh, Hải Phòng, Đà Nẵng và Cần Thơ. Dữ liệu thu thập bao gồm các thông số khí tượng cơ bản như nhiệt độ, độ ẩm, áp suất khí quyển, tốc độ gió và điều kiện thời tiết. Sau khi thu thập, dữ liệu được đóng gói dưới dạng thông điệp JSON và gửi vào các Kafka Topic tương ứng với từng thành phố để đảm bảo phân tách luồng dữ liệu.

Kafka Consumer lắng nghe các Topic và thực hiện các bước chuẩn hóa dữ liệu trước khi lưu trữ, bao gồm:

- Chuyển đổi dữ liệu từ định dạng JSON sang CSV.
- Chuẩn hóa trường thời gian và định dạng dữ liệu.
- Kiểm tra tính hợp lệ và loại bỏ các bản ghi thiếu hoặc sai định dạng.

Việc sử dụng Kafka cho phép tách biệt quá trình thu thập và xử lý dữ liệu, đồng thời đảm bảo khả năng mở rộng và độ tin cậy của pipeline khi khối lượng dữ liệu tăng lên.

4.3.2. Giai đoạn 2: Tích hợp và lưu trữ (Storage - Ghi vào MinIO Bronze Layer)

Sau khi được xử lý tại Kafka Consumer, dữ liệu được chuyển sang hệ thống lưu trữ đối tượng MinIO và tổ chức theo mô hình Data Lake. Trong nghiên cứu này, dữ liệu được lưu tại tầng Bronze (Bronze Layer), đóng vai trò lưu trữ dữ liệu thô đã được chuẩn hóa ở mức cơ bản.

Dữ liệu được tổ chức theo phân vùng không gian và thời gian. Cụ thể:

- Dữ liệu lịch sử từ năm 2000 đến thời điểm hiện tại được hợp nhất thành một tập dữ liệu tổng hợp để phục vụ huấn luyện mô hình.

- Dữ liệu mới thu thập định kỳ từ API được lưu dưới dạng các tệp riêng lẻ theo chu kỳ thời gian, nhằm hỗ trợ xử lý gia tăng và cập nhật liên tục.

Bronze Layer đóng vai trò lưu giữ dữ liệu gốc, đảm bảo khả năng truy vết và tái xử lý khi cần thiết. Đây là nền tảng để triển khai các tầng xử lý tiếp theo như Silver Layer (dữ liệu đã làm sạch) và Gold Layer (dữ liệu phục vụ phân tích và mô hình hóa).

Hai giai đoạn thu thập và lưu trữ dữ liệu tạo thành pipeline dữ liệu ban đầu của hệ thống. Thiết kế này đảm bảo dữ liệu được thu thập từ nguồn, xử lý trung gian qua Kafka và lưu trữ tập trung trên MinIO, làm cơ sở cho các bước tiền xử lý và xây dựng mô hình dự báo trong các giai đoạn tiếp theo.

4.3.3. Giai đoạn 3: Xử lý và làm sạch dữ liệu (Processing - Spark Transformations)

Mục tiêu:

- Chuẩn bị dữ liệu sạch, đầy đủ và có cấu trúc để phục vụ cho việc huấn luyện mô hình học máy.
- Xử lý các vấn đề trong dữ liệu thô như: missing values, outliers, chuẩn hóa dữ liệu.
- Tạo ra các đặc trưng (features) mới từ dữ liệu gốc để nâng cao hiệu suất dự đoán của mô hình.
- Áp dụng các phép biến đổi Spark để xử lý dữ liệu lớn hiệu quả và song song.

Các bước thực hiện:

1. Chuẩn bị môi trường và import thư viện:
    - Sử dụng PySpark để xử lý dữ liệu lớn.
    - Import các thư viện cần thiết: SparkSession, Window, các hàm như col, lag, avg, stddev, max, min, date_format, row_number, và các transformer như VectorAssembler, MinMaxScaler, StandardScaler.
2. Đọc dữ liệu từ CSV:

- Dữ liệu được tải từ Google Drive (file Vietnam_IoT_Final_Dataset.csv).
- Đọc vào Spark DataFrame với schema tự động suy luận.

3. Khám phá dữ liệu (Data Exploration):
   - Kiểm tra tổng số dòng (1,143,360) và số cột (17).
   - Phát hiện cột visibility có giá trị thiếu (1,143,360 missing values).
   - Phân tích thống kê mô tả các biến số học (temperature, humidity, pressure, rain, wind_speed, etc.).
   - Phân tích biến phân loại (weather_code, region, is_day).
4. Xử lý giá trị thiếu và chuẩn hóa dữ liệu:
   - Loại bỏ hoặc xử lý cột visibility do thiếu dữ liệu hoàn toàn.
   - Chuẩn hóa dữ liệu số học bằng StandardScaler hoặc MinMaxScaler để đưa về cùng thang đo.
5. Tạo đặc trưng (Feature Engineering):
   - Tạo các đặc trưng mới dựa trên timestamp (ngày, tháng, giờ, mùa, etc.).
   - Tạo các đặc trưng thống kê trượt (moving average, std, min, max) cho từng sensor theo thời gian.
   - Sử dụng Window functions để tính toán các đặc trưng theo thời gian.
6. Chuẩn bị dữ liệu cho mô hình:
   - Ghép các đặc trưng thành vector bằng VectorAssembler.
   - Chia dữ liệu thành tập huấn luyện và tập kiểm tra theo thời gian (TimeSeriesSplit).

4.3.4. Giai đoạn 4: Xây dựng và huấn luyện mô hình dự đoán (Machine Learning)

Mục tiêu:

- Xây dựng các mô hình học máy có khả năng dự đoán chính xác các chỉ số thời tiết (temperature, humidity, rain, etc.).
- So sánh hiệu suất của nhiều thuật toán để lựa chọn mô hình tốt nhất.

- Tối ưu hóa siêu tham số để cải thiện độ chính xác của mô hình.
- Đánh giá mô hình một cách khách quan và chuẩn bị cho việc triển khai thực tế.
- Lưu trữ mô hình tốt nhất để tái sử dụng mà không cần huấn luyện lại.

Các bước thực hiện:

1. Lựa chọn mô hình:
    - Sử dụng các mô hình hồi quy và phân loại phổ biến:
        - RandomForestRegressor
        - LightGBM (Light Gradient Boosting Machine)
        - XGBoost (Extreme Gradient Boosting)
2. Huấn luyện mô hình:
    - Sử dụng TimeSeriesSplit để chia dữ liệu theo thời gian, tránh leak dữ liệu.
    - Huấn luyện từng mô hình trên tập huấn luyện.
    - Tối ưu siêu tham số bằng GridSearchCV hoặc RandomizedSearchCV.
3. Đánh giá mô hình:
    - Sử dụng các chỉ số đánh giá:
        - Mean Squared Error (MSE)
        - Root Mean Squared Error (RMSE)
        - Mean Absolute Error (MAE)
        - R^2 Score
    - So sánh hiệu suất giữa các mô hình.
4. Dự đoán và kiểm tra:
    - Dự đoán trên tập kiểm tra.
    - Trực quan hóa kết quả dự đoán so với thực tế bằng matplotlib và seaborn.
5. Lưu mô hình:
    - Lưu mô hình tốt nhất bằng joblib hoặc pickle để sử dụng sau này.

4.3.5. Giai đoạn 5: Trực quan hóa và báo cáo (Visualization - Report)

Sau khi hoàn thành các giai đoạn thu thập dữ liệu, xử lý, xây dựng mô hình và dự đoán, giai đoạn cuối cùng tập trung vào việc trực quan hóa kết quả và trình bày báo cáo để giúp người dùng dễ dàng theo dõi, phân tích và đưa ra quyết định. Trong dự án này, nhóm đã sử dụng Streamlit để xây dựng một giao diện trực quan, thân thiện với người dùng và có tính tương tác cao.

4.3.5.1. Trực quan hóa dự đoán nhiệt độ theo thời gian

Trong tính năng "Dự báo nhiệt độ", người dùng có thể chọn một thành phố cụ thể (ví dụ: Cần Thơ) để xem kết quả dự báo nhiệt độ cho từng thời điểm trong tương lai.

Kết quả được trình bày trong một bảng dữ liệu, bao gồm các thông tin sau:

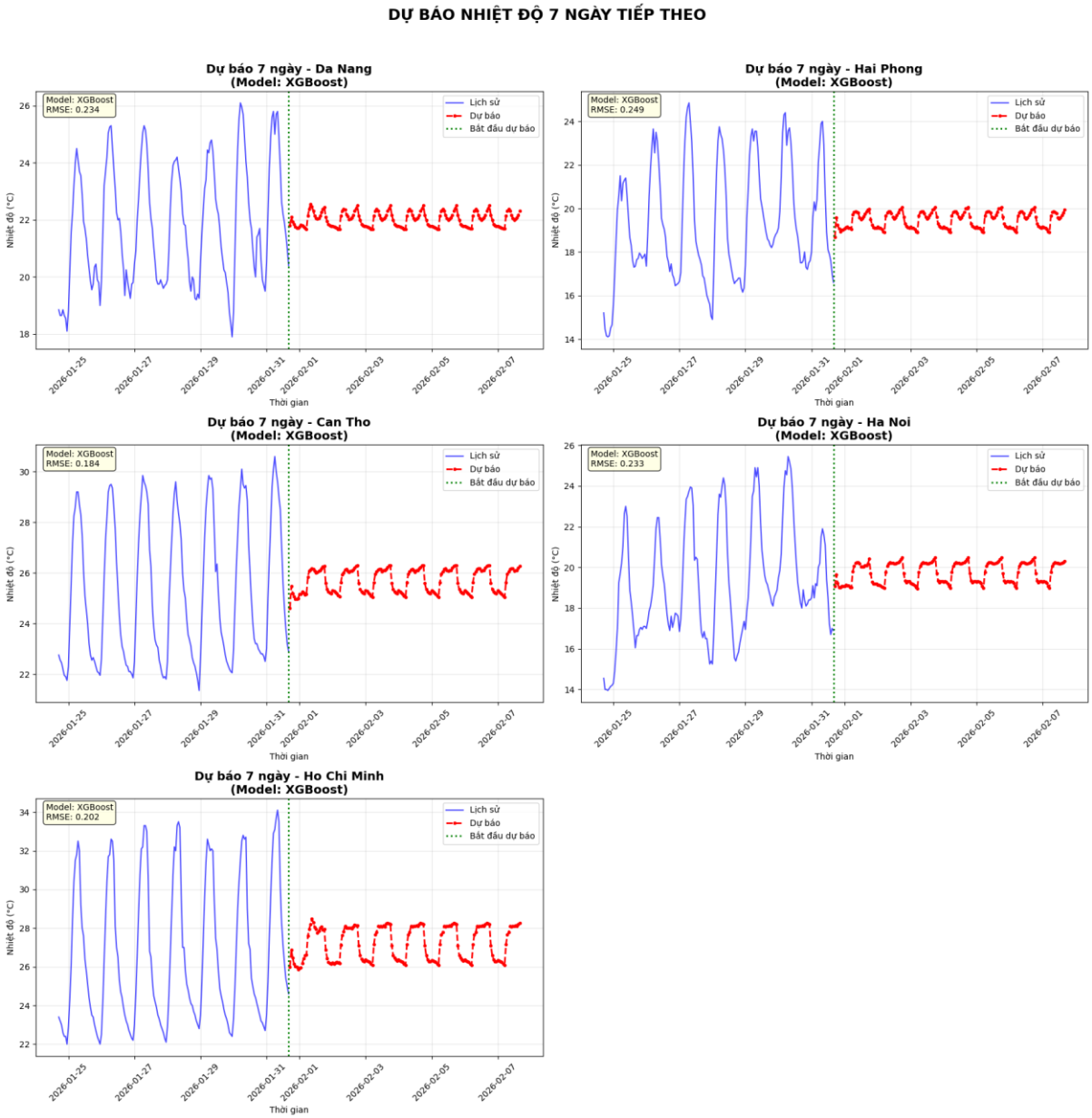
- Thời gian (ngày, giờ)
- Nhiệt độ dự báo (temp)

Ngoài ra, hệ thống còn hiển thị biểu đồ đường thể hiện sự biến động nhiệt độ theo thời gian.

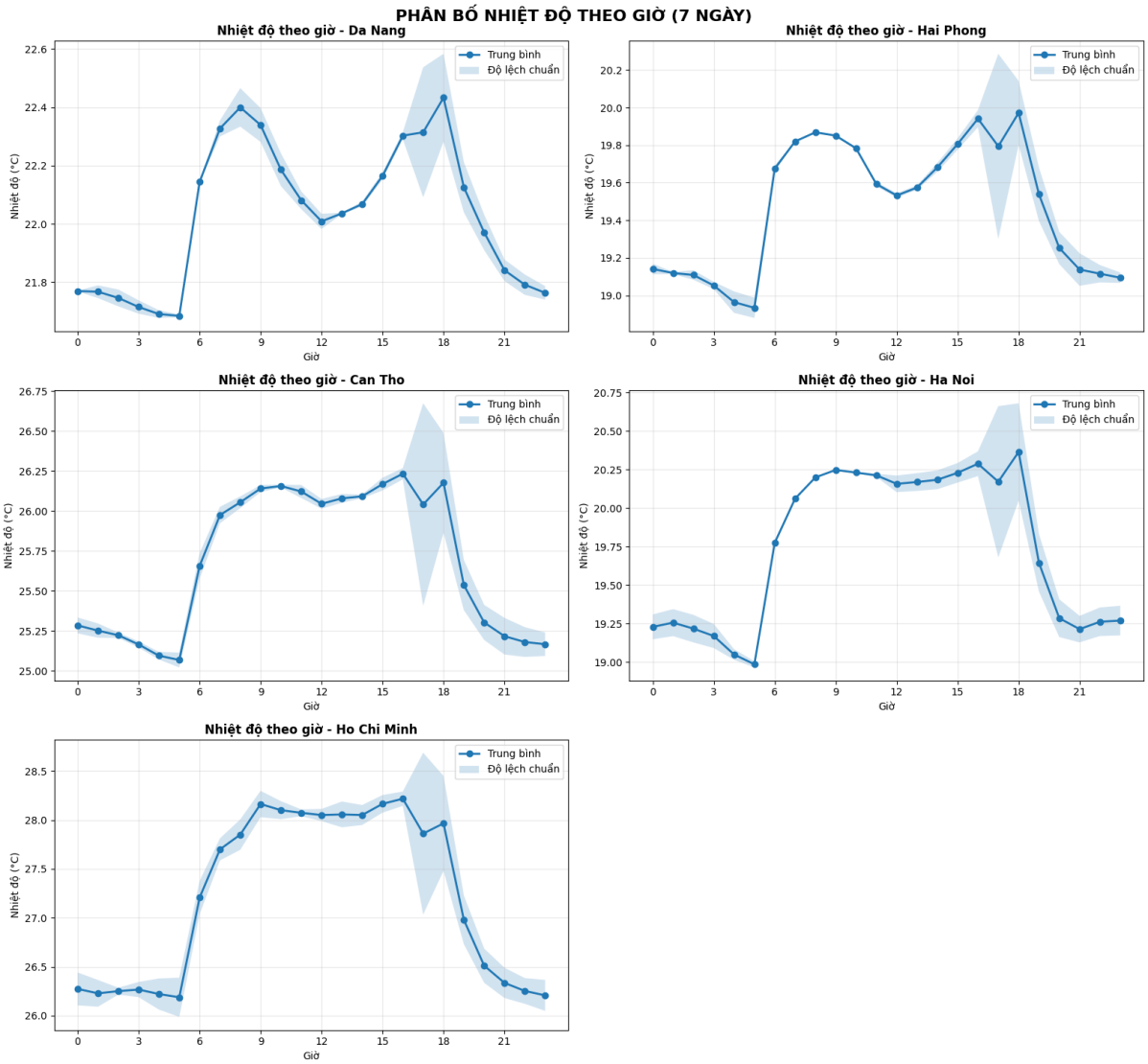
Biểu đồ này giúp người dùng:

- Quan sát xu hướng tăng/giảm nhiệt độ
- So sánh nhiệt độ giữa các thời điểm khác nhau trong ngày
- Nhanh chóng nắm bắt đặc điểm thời tiết của thành phố đã chọn.

Sự kết hợp giữa bảng dữ liệu và biểu đồ giúp tăng cường tính trực quan, đồng thời đảm bảo cả chi tiết lẫn cái nhìn tổng quan toàn diện.



Hình 11. Trực quan hóa dự đoán nhiệt độ theo ngày



Hình 12. Trực quan hóa dự đoán nhiệt độ theo giờ

4.3.5.2. Trực quan hóa các đề xuất thành phố du lịch

Trong tính năng "Đề xuất thành phố", hệ thống giúp người dùng chọn mức nhiệt độ mong muốn (ví dụ: Lạnh ≤ 20°C). Dựa trên kết quả dự báo nhiệt độ trung bình cho mỗi thành phố vào ngày hôm sau, hệ thống sẽ thực hiện so sánh và xếp hạng.

Kết quả được trình bày như sau:

- Bảng so sánh thể hiện nhiệt độ trung bình dự báo giữa các thành phố (Cần Thơ, Hà Nội, Hồ Chí Minh, Hải Phòng, Đà Nẵng)

- Điểm đánh giá mức độ phù hợp để xác định mức độ phù hợp của mỗi thành phố với sở thích thời tiết của người dùng.

Thành phố phù hợp nhất được đánh dấu rõ ràng, kèm theo thông báo kết quả và nhiệt độ trung bình dự báo.

Ngoài ra, hệ thống sử dụng biểu đồ cột để so sánh nhiệt độ trung bình giữa các thành phố, giúp người dùng:

- Dễ dàng nhận biết sự khác biệt về nhiệt độ
- Có cái nhìn trực quan trước khi lựa chọn điểm đến

4.3.5.3. Báo cáo kết quả và hỗ trợ ra quyết định

Giao diện Streamlit hoạt động như một báo cáo tương tác, cho phép người dùng:

- Tương tác trực tiếp với dữ liệu (chọn thành phố, chọn mức nhiệt độ)
- Quan sát kết quả ngay lập tức mà không cần xử lý thủ công.
- Xem sự kết hợp giữa dữ liệu số, biểu đồ và kết luận khuyến nghị.

Do đó, giai đoạn trực quan hóa và báo cáo không chỉ giúp trình bày kết quả của mô hình một cách rõ ràng mà còn nâng cao trải nghiệm người dùng và tính ứng dụng thực tiễn của hệ thống.

CHƯƠNG V: THỰC NGHIỆM VÀ KẾT QUẢ

5.1. Kết quả vận hành hệ thống

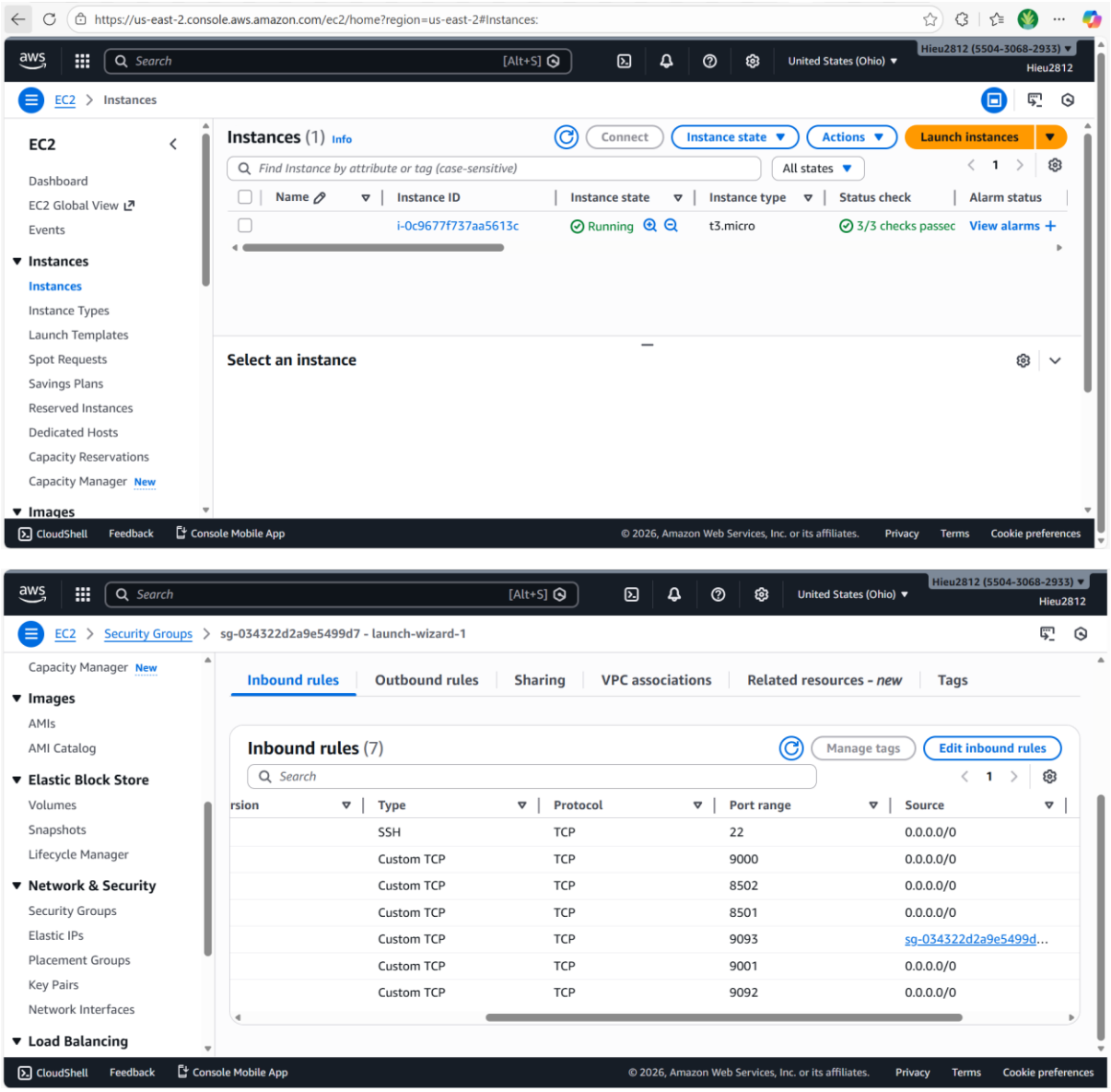
Sau quá trình thiết kế, triển khai và kiểm thử, hệ thống dự đoán thời tiết và đề xuất thành phố phù hợp để du lịch đã được vận hành thành công trên môi trường Amazon EC2. Kết quả vận hành cho thấy hệ thống hoạt động ổn định, các thành phần phối hợp hiệu quả và đáp ứng được các yêu cầu đề ra ban đầu.

Cụ thể, hệ thống đã thực hiện tốt các chức năng sau:

- Thu thập dữ liệu thời tiết: Dữ liệu được lấy tự động từ API Open-Meteo theo chu kỳ xác định. Quá trình thu thập diễn ra liên tục và ổn định, đảm bảo dữ liệu đầu vào luôn được cập nhật.
- Truyền dữ liệu theo luồng: Apache Kafka hoạt động hiệu quả trong vai trò trung gian tiếp nhận và phân phối dữ liệu thời tiết. Dữ liệu được đẩy vào các topic và tiêu thụ bởi các consumer mà không xảy ra tình trạng mất dữ liệu.
- Xử lý dữ liệu lớn: PySpark đảm nhiệm việc làm sạch, tổng hợp và xử lý dữ liệu thời tiết. Kết quả cho thấy thời gian xử lý phù hợp với khối lượng dữ liệu thu thập được, đáp ứng yêu cầu xử lý theo lô và gần thời gian thực.
- Lưu trữ dữ liệu: Dữ liệu thô và dữ liệu sau xử lý được lưu trữ trên MinIO, đảm bảo khả năng truy xuất nhanh và phục vụ cho các bước phân tích và trực quan hóa sau này.
- Hiển thị và đề xuất: Giao diện Streamlit hiển thị thông tin dự báo thời tiết và danh sách các thành phố được đề xuất cho du lịch một cách trực quan, dễ sử dụng. Người dùng có thể theo dõi xu hướng thời tiết và lựa chọn điểm đến phù hợp.

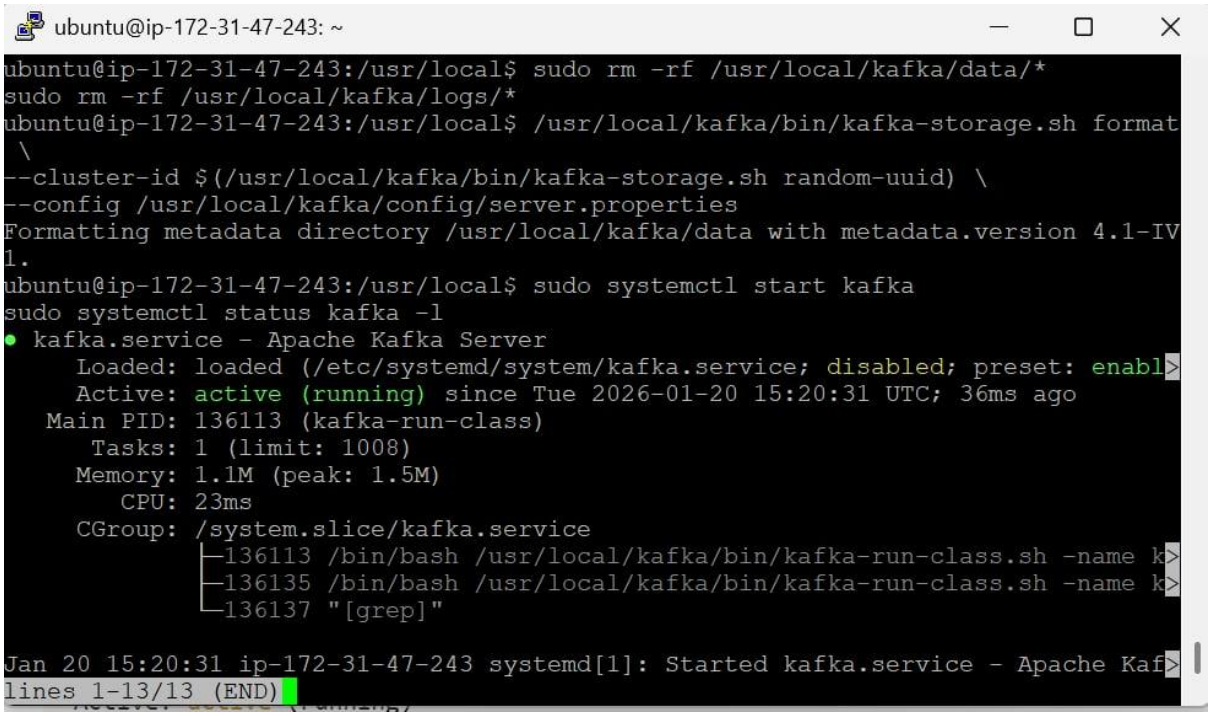
Trong quá trình vận hành thử nghiệm, hệ thống không ghi nhận lỗi nghiêm trọng, các thành phần hoạt động đồng bộ và ổn định. Kết quả cho thấy kiến trúc hệ thống là phù hợp cho bài toán xử lý dữ liệu thời tiết và có khả năng mở rộng trong tương lai khi khối lượng dữ liệu hoặc số lượng người dùng tăng lên.

EC2:



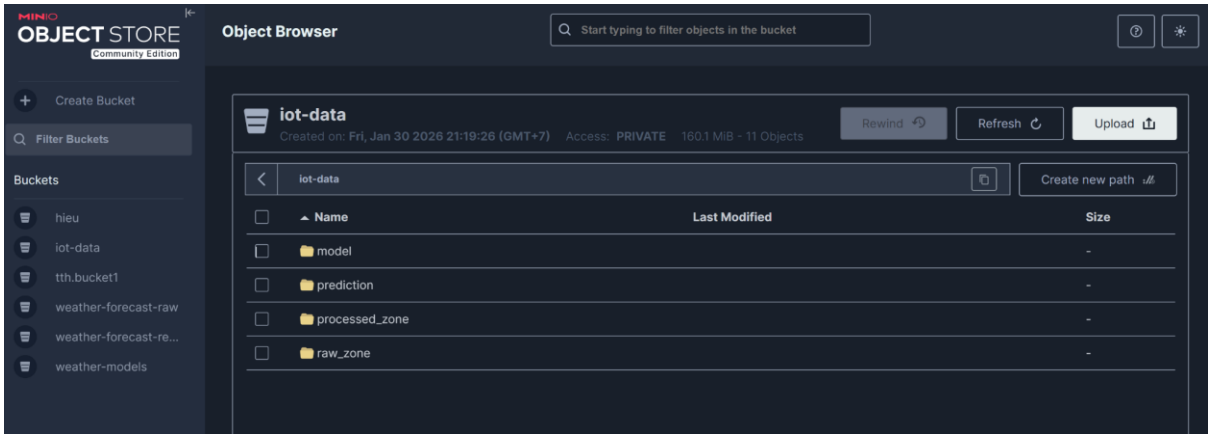
Hình 13. Kết quả vận hành trên EC2

Kafka:



Hình 14. Kết quả chạy thành công trên Kafka

MinIO:



Hình 15. Kết quả vận hàng trên MinIo

5.2. Kết quả phân tích và dự báo

Dựa trên hệ thống xử lý dữ liệu thời tiết thời gian thực được xây dựng với Apache Kafka, Apache Spark và Data Lake MinIO, nhóm đã tiến hành phân tích dữ liệu và thực hiện dự báo nhiệt độ cho các thành phố nghiên cứu. Kết quả thu được cho thấy hệ thống có khả năng khai thác hiệu quả dữ liệu lớn và cung cấp thông tin dự báo có ý nghĩa thực tiễn.

5.2.1. Kết quả phân tích dữ liệu thời tiết

Thông qua quá trình xử lý và phân tích dữ liệu bằng PySpark, hệ thống đã rút ra một số đặc điểm nổi bật của dữ liệu thời tiết:

- Nhiệt độ thể hiện rõ tính chu kỳ theo thời gian, đặc biệt là chu kỳ ngày – đêm. Nhiệt độ thường đạt giá trị cao nhất vào buổi trưa và đầu giờ chiều, trong khi giảm thấp vào ban đêm và sáng sớm.
- Sự khác biệt nhiệt độ giữa các thành phố được thể hiện rõ ràng, phản ánh đặc trưng khí hậu của từng vùng miền:
    - Hà Nội và Hải Phòng có xu hướng nhiệt độ thấp hơn vào một số thời điểm, đặc biệt trong mùa lạnh.
    - Đà Nẵng có mức nhiệt độ trung bình ổn định hơn.
    - Cần Thơ có nhiệt độ trung bình cao hơn, mang đặc trưng khí hậu nhiệt đới.
- Các thống kê tổng hợp như nhiệt độ trung bình theo giờ, theo ngày được hệ thống tính toán chính xác và lưu trữ phục vụ cho bước dự báo và trực quan hóa.

Kết quả phân tích cho thấy dữ liệu sau khi được làm sạch và xử lý có chất lượng tốt, đủ điều kiện để sử dụng trong các mô hình dự báo ngắn hạn.

5.2.2. Kết quả dự báo nhiệt độ

Dựa trên dữ liệu lịch sử và các đặc trưng theo thời gian, mô hình dự báo đã được áp dụng để dự đoán nhiệt độ theo từng mốc giờ trong tương lai cho từng thành phố.

- Kết quả dự báo thể hiện:
    - Xu hướng nhiệt độ ổn định, không xuất hiện các giá trị bất thường lớn.
    - Sự thay đổi nhiệt độ theo thời gian vẫn tuân theo quy luật tự nhiên đã quan sát trong dữ liệu lịch sử.
- Các giá trị nhiệt độ dự đoán được trình bày chi tiết theo:
    - Ngày và giờ
    - Nhiệt độ dự đoán tương ứng

Kết quả cho thấy mô hình có khả năng phản ánh tương đối chính xác diễn biến nhiệt độ trong ngắn hạn, phù hợp với mục tiêu minh họa năng lực xử lý và dự báo của hệ thống Big Data thời gian thực.

5.2.3. Kết quả đề xuất thành phố dựa trên dự báo

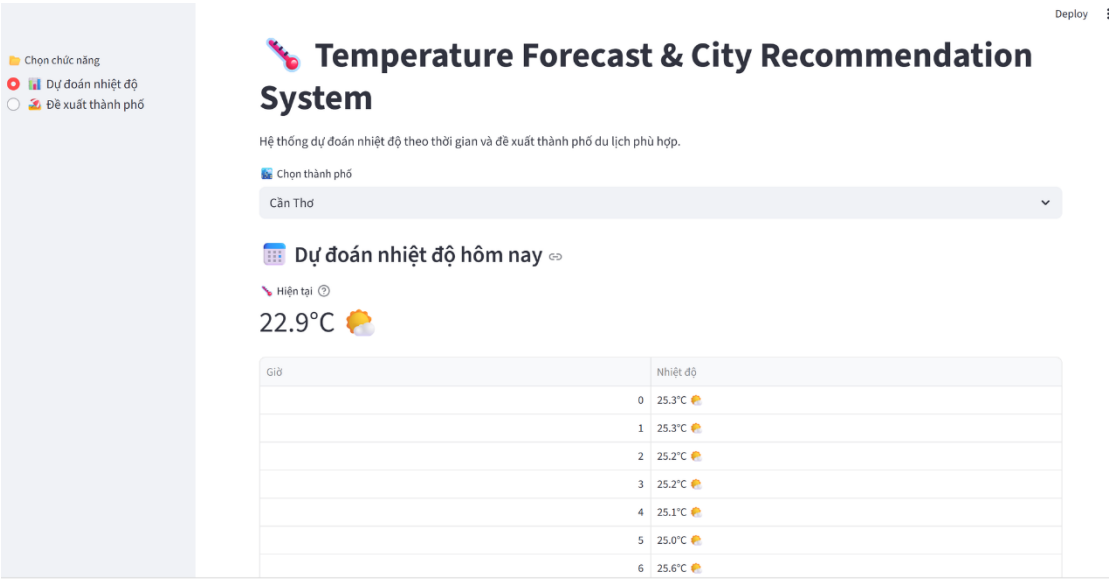
Từ kết quả dự báo nhiệt độ, hệ thống tiếp tục thực hiện phân tích so sánh giữa các thành phố để phục vụ chức năng đề xuất điểm đến phù hợp.

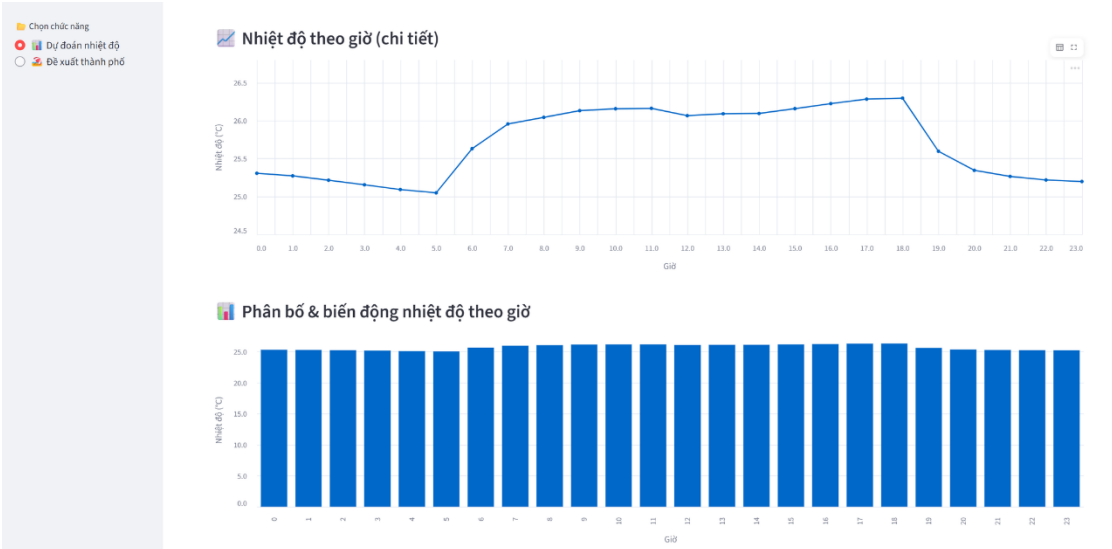
- Nhiệt độ trung bình dự đoán của từng thành phố cho ngày hôm sau được tính toán và tổng hợp.
- Dựa trên mức nhiệt độ mong muốn do người dùng lựa chọn (ví dụ: lạnh ≤ 20°C), hệ thống:
    - Gán điểm đánh giá (score) cho từng thành phố
    - Xếp hạng mức độ phù hợp
- Thành phố có nhiệt độ dự đoán đáp ứng tốt nhất tiêu chí được hệ thống đề xuất làm lựa chọn tối ưu.

Kết quả đề xuất cho thấy hệ thống không chỉ dừng lại ở việc dự báo số liệu, mà còn chuyển hóa dữ liệu thành thông tin có giá trị hỗ trợ ra quyết định, đặc biệt trong các bài toán liên quan đến du lịch và sinh hoạt.

5.3. Kết quả trực quan hóa

Streamlit:





Temperature Forecast & City Recommendation System

Hệ thống dự đoán nhiệt độ theo thời gian và đề xuất thành phố du lịch phù hợp.

Đề xuất thành phố du lịch (hôm nay)

Bạn thích thời tiết như thế nào?

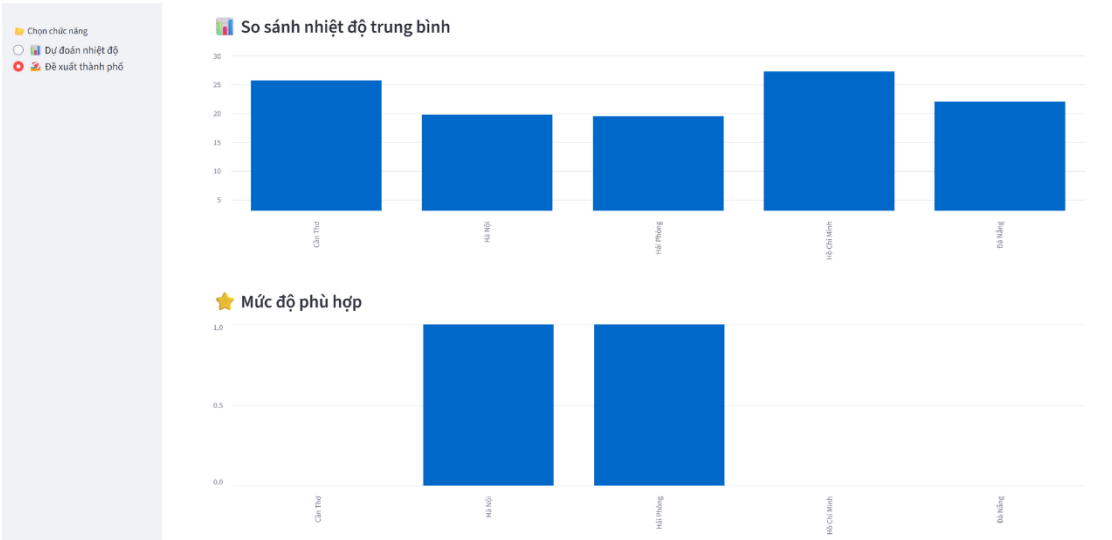
Lạnh (≤ 20°C)

city	avg_temp	score
Hà Nội	19.8°C	1
Hải Phòng	19.5°C	1
Cần Thơ	25.7°C	0
Đà Nẵng	22.0°C	0
Hồ Chí Minh	27.3°C	0

Thành phố phù hợp nhất hôm nay: Hà Nội (19.8°C)

So sánh nhiệt độ trung bình





Hình 16. Kết quả trực quan hóa

5.4. Đánh giá hiệu năng

Hiệu năng của hệ thống được đánh giá thông qua quá trình vận hành thử nghiệm trên môi trường EC2 với dữ liệu thời tiết thu thập từ API Open-Meteo. Kết quả cho thấy Apache Kafka hoạt động ổn định trong việc tiếp nhận và truyền dữ liệu theo luồng, không xảy ra tình trạng mất dữ liệu, đồng thời đảm bảo độ trễ thấp.

Các job xử lý dữ liệu bằng PySpark vận hành hiệu quả, đáp ứng tốt yêu cầu xử lý gần thời gian thực. Việc xử lý dữ liệu theo mô hình phân tán giúp rút ngắn thời gian xử lý và cho thấy khả năng mở rộng khi tăng số lượng node xử lý.

Bên cạnh đó, MinIO đáp ứng tốt nhu cầu lưu trữ và truy xuất dữ liệu, trong khi giao diện Streamlit hiển thị dữ liệu trực quan và cập nhật kịp thời. Nhìn chung, hệ thống đạt hiệu năng phù hợp với phạm vi đồ án và có khả năng phát triển trong các ứng dụng thực tế.

CHƯƠNG VI: KẾT LUẬN VÀ KIẾN NGHỊ

6.1. Kết quả đạt được

Đồ án đã xây dựng thành công một hệ thống Big Data phục vụ cho bài toán dự đoán thời tiết và đề xuất thành phố phù hợp để du lịch, sử dụng nguồn dữ liệu thời tiết từ API Open-Meteo. Hệ thống được triển khai trên hạ tầng Amazon EC2 và tích hợp các công nghệ xử lý dữ liệu lớn hiện đại nhằm đảm bảo khả năng xử lý dữ liệu liên tục và mở rộng linh hoạt.

Các kết quả chính đạt được bao gồm:

- Kết nối và thu thập dữ liệu thời tiết thời gian thực từ API Open-Meteo, đảm bảo dữ liệu đa dạng và cập nhật liên tục.
- Xây dựng pipeline xử lý dữ liệu sử dụng Apache Kafka để tiếp nhận và truyền dữ liệu thời tiết theo luồng.
- Sử dụng PySpark để xử lý, làm sạch, tổng hợp dữ liệu và thực hiện dự đoán các chỉ số thời tiết quan trọng như nhiệt độ, độ ẩm và lượng mưa.
- Lưu trữ dữ liệu thô và dữ liệu đã xử lý trên MinIO, đóng vai trò như một hệ thống data lake cho toàn bộ ứng dụng.
- Phát triển giao diện trực quan bằng Streamlit, giúp người dùng dễ dàng theo dõi dự báo thời tiết và nhận đề xuất các thành phố phù hợp cho hoạt động du lịch.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định trên môi trường EC2, có khả năng xử lý dữ liệu lớn và cung cấp các đề xuất du lịch dựa trên điều kiện thời tiết một cách trực quan và dễ sử dụng.

6.2. Hạn chế của đề tài

Bên cạnh những kết quả đạt được, đồ án vẫn còn tồn tại một số hạn chế như sau:

- Dữ liệu thu thập từ Open-Meteo API chủ yếu là dữ liệu dự báo và lịch sử thời tiết, chưa phản ánh đầy đủ các yếu tố thời tiết cực đoan hoặc bất thường.
- Độ chính xác của kết quả dự đoán còn phụ thuộc nhiều vào mô hình xử lý và độ chi tiết của dữ liệu đầu vào.

- Hệ thống đề xuất thành phố du lịch hiện mới chỉ dựa trên các tiêu chí thời tiết cơ bản, chưa kết hợp các yếu tố khác như chi phí du lịch, lượng khách, hoặc các sự kiện văn hóa – xã hội.
- Quy mô triển khai còn giới hạn, hệ thống chưa được kiểm thử với khối lượng dữ liệu rất lớn hoặc số lượng người dùng đồng thời cao.
- Giao diện Streamlit tuy trực quan nhưng còn đơn giản, chưa hỗ trợ nhiều chức năng nâng cao như cá nhân hóa hay phân tích sâu theo nhu cầu người dùng.

6.3. Hướng phát triển trong tương lai

Trong tương lai, đề tài có thể tiếp tục được phát triển và mở rộng theo các hướng sau:

- Nâng cao độ chính xác của mô hình dự đoán bằng cách áp dụng các kỹ thuật học máy và học sâu cho dữ liệu chuỗi thời gian như LSTM, GRU hoặc Transformer.
- Mở rộng nguồn dữ liệu bằng cách kết hợp thêm các API thời tiết khác hoặc dữ liệu vệ tinh nhằm tăng độ tin cậy của kết quả dự báo.
- Bổ sung thêm các tiêu chí đề xuất thành phố du lịch như chi phí, mức độ an toàn, mật độ du khách và đánh giá từ người dùng.
- Phát triển hệ thống gợi ý du lịch cá nhân hóa dựa trên sở thích, thời gian và ngân sách của từng người dùng.
- Triển khai hệ thống trên môi trường cloud với quy mô lớn hơn, tối ưu hiệu năng và khả năng chịu tải để phục vụ ứng dụng thực tế.

TÀI LIỆU THAM KHẢO

[1]. Amazon Web Services, Inc. (2023). *Amazon EC2 instance types*. https://aws.amazon.com/ec2/instance-types/

[2]. MinIO, Inc. (2023). *MinIO object storage for Kubernetes*. https://min.io/docs/minio/kubernetes/upstream/index.html

[3]. The Apache Software Foundation. (2023). *Apache Kafka documentation*. https://kafka.apache.org/documentation/

[4]. The Apache Software Foundation. (2023). *Apache Spark documentation*. https://spark.apache.org/docs/latest/

[5]. Streamlit, Inc. (2023). *Streamlit documentation*. https://docs.streamlit.io/

PHỤ LỤC

Link các file trong đồ án: BigData_Group6_FinalProject - Google Drive

Link stream-lit app: http://18.219.196.163:8502/

Link minIO: http://18.219.196.163:9001/

Link kafka: http://18.219.196.163:9092/

PHẦN 2. MÔ TẢ PHÂN CÔNG CÔNG VIỆC TRONG NHÓM

Mã số sinh viên	Họ và tên	Mức độ đóng góp	Nội dung thực hiện
K224060785	Trần Trung Hiếu (Nhóm trưởng)	100%	Triển khai kafka, deploy lên awc ec2, code stream-lit app, viết báo cáo word
K224060772	Nguyễn Thị Hồng Diễm	100%	Tìm kiếm data source, xây dựng và huấn luyện mô hình dự đoán, viết báo cáo word
K224060779	Nguyễn Thị Như Hào	100%	Tìm kiếm data source, đẩy dataset lên MinIO, viết báo cáo word
K224060822	Nguyễn Phương Tuyền	100%	Code stream-lit app, viết báo cáo word
K224060825	Trần Thị Hiếu Xuân	100%	Xây dựng và huấn luyện mô hình dự đoán, viết báo cáo word